[Extended Version] Pacha Sketches: Organizing Multidimensional Space for Efficient Aggregation

Rudi Poepsel-Lemaitre
BIFOLD & TU Berlin
Berlin, Germany
r.poepsellemaitretu-berlin.de

Kaustubh Beedkar
Indian Institute of Technology Delhi
Delhi, India
kbeedkar@cse.iitd.ac.in

Volker Markl
BIFOLD, TU Berlin & DFKI
Berlin, Germany
volker.markl@tu-berlin.de

Notice. This document is an extended version of the paper published at IEEE ICDE 2027. Please cite the original IEEE ICDE 2027 publication when referencing this work. As this version builds upon an earlier manuscript, some legacy errors or inconsistencies may remain. If you encounter any issues or have questions about the content, please do not hesitate to contact Rudi Poepsel-Lemaitre.

***Abstract*—Aggregating data items over subpopulations of a multidimensional space is a fundamental operation in applications such as statistical analysis, query optimization, and machine learning, but evaluating such queries at scale is infeasible without data summarization. In this paper, we introduce a principled hierarchical organization that distinguishes between categorical and numerical dimensions to organize the multidimensional space effectively. For numerical attributes, we extend dyadic ranges from one-dimensional to multi-dimensional domains, while for categorical attributes, we leverage AD-trees to efficiently identify and summarize categorical regions. Together, these components enable scalable and composable summarization of multidimensional data. We instantiate this organization in a novel data summarization technique, called PACHA SKETCHES. PACHA SKETCHES are designed to be mergeable, supporting efficient distributed computation in both federated and streaming settings. Moreover, PACHA SKETCHES are modular, allowing easy adaptation to different workloads, support for new aggregate functions, and extensions that provide formal privacy guarantees. Extensive experiments on real-world and synthetic datasets demonstrate that PACHA SKETCHES consistently achieve high accuracy and efficiency, even in high-dimensional settings with more than ten attributes and millions of tuples. Under equivalent space budgets, PACHA SKETCHES outperform state-of-the-art Omni Sketches by up to three orders of magnitude in estimation accuracy and two orders of magnitude in query performance.**

***Index Terms*—data summarization, approximate query processing, multidimensional data.**

I. INTRODUCTION

Modern data-intensive applications routinely generate and analyze *multidimensional data*, in which multiple attributes describe each record. This data model arises naturally across a wide range of domains, including e-commerce, sensor networks, healthcare, and online services. A fundamental analytical task over such data is to compute aggregates over subsets of records defined by combinations of attribute predicates. We refer to such tasks as *multidimensional aggregate queries*. For example, an analyst may request the number of events occurring within a geographic region over a specific time interval, or the total transaction volume for a given product category and customer age range. These queries underpin many downstream applications, including trend analysis [1], [2], anomaly detection [3], [4], user behavior modeling [5], and capacity planning [6], [7].

Evaluating multidimensional aggregate queries exactly is often infeasible in practice. At scale, such queries require repeated scans over massive datasets, incurring prohibitive computational costs. In federated environments, raw data is distributed across administrative domains and cannot be centrally stored or repeatedly accessed due to privacy or policy constraints [8]–[10]. In streaming settings, data arrives continuously and must be processed with bounded memory and low latency, precluding exact aggregation over the full data history [11], [12]. Traditional multidimensional indexing and OLAP techniques, such as Quad-trees, Octrees, and data cubes, also face severe scalability limitations in high-dimensional settings, as their storage and maintenance costs grow exponentially with the number of dimensions and with increasing data sparsity [13]–[15]. As a result, data management systems increasingly rely on *summaries* that enable approximate aggregate computation without accessing the full dataset [16]. Designing such summaries for multidimensional aggregate queries is therefore a central problem in modern data management.

Summarizing multidimensional data poses fundamental challenges. First, real-world datasets often involve tens of attributes, and analytical workloads must support queries over arbitrary subsets of them. The number of possible attribute combinations grows exponentially with dimensionality, rendering explicit representations of joint distributions impractical. Consequently, traditional approaches such as multidimensional histograms [17] and early sketch-based techniques [11], [18], [19] struggle to scale beyond low-dimensional settings. Second, multidimensional data typically consists of heterogeneous attribute types. Categorical attributes (e.g., product category or region) coexist with numerical attributes (e.g., price or age), and queries combine equality predicates with numerical range constraints. Most existing summarization techniques are designed for a single attribute type or predicate class, and extending them to mixed-type, high-dimensional queries

often incurs prohibitive space overheads or severe accuracy degradation.

At a deeper level, these challenges stem from the absence of a principled way to *organize mixed-type multidimensional spaces*. Existing approaches largely treat the domain as a flat Cartesian product of attributes or discretize each dimension independently, ignoring the fundamentally different structural properties of categorical and numerical domains. As a result, they fail to expose a reusable hierarchical representation of the multidimensional space—-one that could support efficient aggregation, composability across queries, and reuse across workloads. This suggests that the core challenge is not merely how to summarize data, but how to structure the multidimensional domain itself to enable scalable aggregation.

Prior work on multidimensional summaries includes sampling-based methods [20]–[24], multidimensional histograms [25], [26], wavelet-based techniques [27], and sketch-based approaches [28]–[30]. Sampling-based methods, while simple, suffer from high variance for selective or high-dimensional queries and require storing raw samples, which may be infeasible in privacy-sensitive or federated settings [8]–[10]. Sketch-based techniques provide compact and mergeable summaries with strong theoretical guarantees, but most incur exponential space or time costs when extended to higher dimensions [28], [29]. More recent approaches, such as Omni Sketches [30], combine sketching with sampling to improve accuracy for multidimensional queries, yet their performance degrades for range predicates and for queries involving many conditions. More fundamentally, these methods do not provide an explicit hierarchical organization of mixed-type dimensions, limiting their scalability and generality.

In this paper, we introduce a principled hierarchical organization of mixed-type multidimensional spaces that explicitly distinguishes between categorical and numerical dimensions. Our approach decomposes the multidimensional domain into complementary structural components: an AD-tree [31] over categorical attributes that hierarchically captures conjunctions of categorical values, and a $\mathbb{B}$-adic space cover over numerical dimensions that extends dyadic intervals [11], [18] to support multidimensional range predicates. This decomposition yields a reusable structural abstraction for scalable multidimensional computation.

As a concrete instantiation of this organization, we propose PACHA SKETCHES,[1] a unified, mergeable summary structure for multidimensional aggregate queries. PACHA SKETCHES supports equality predicates over categorical attributes and range predicates over numerical attributes, enabling compact summarization of joint distributions across dozens of mixed-type dimensions. By leveraging hierarchical covers, PACHA SKETCHES supports interactive OLAP-style exploration and operates efficiently in a streaming setting.

Each PACHA SKETCHES instance consists of a collection of base sketches for point-query estimation (e.g., Count-Min sketches [11]), indexed at every level of the $\mathbb{B}$-adic space cover, together with lightweight indexing structures such as Bloom filters [32] to prune empty regions in sparse multidimensional spaces. On insertion, tuples traverse the categorical AD-tree while simultaneously identifying the relevant $\mathbb{B}$-adic cubes for numerical attributes, updating only the necessary components. During query processing, the system computes minimal hierarchical covers for categorical and numerical predicates, prunes unpopulated regions, and aggregates estimates from selected components. This design yields logarithmic query complexity per dimension, bounded space overhead, and provable error guarantees.

After reviewing preliminaries in Section II, we make the following contributions:

- We formalize a hierarchical organization of mixed-type multidimensional spaces (Section III) that combines an AD-tree over categorical attributes with a $\mathbb{B}$-adic hierarchical cover over numerical attributes, providing a reusable structural abstraction.
- We introduce PACHA SKETCHES (Section IV), a mergeable summary structure that instantiates this organization to efficiently support multidimensional aggregate queries with mixed-type predicates, and provide theoretical guarantees on accuracy, space, and time complexity.
- We evaluate PACHA SKETCHES on four real-world datasets with up to 11 dimensions and millions of records (Section V), demonstrating up to three orders of magnitude higher accuracy and two orders of magnitude faster query performance than state-of-the-art methods such as Omni Sketches [30] under equivalent space budgets.

II. PRELIMINARIES

We start by discussing the data and the query model that we consider in this paper. We then briefly discuss the key data structures used in PACHA SKETCHES.

A. Data Model

Let $A_1, \ldots, A_D$ be D attributes. Each attribute A_j has a domain $\mathbf{dom}(A_j)$. We partition the attributes into numerical and categorical attributes. For numerical A_j we take $\mathbf{dom}(A_j) = \mathbb{R}$ (or a bounded sub-interval of $\mathbb{R}$ determined by observed extremes). For categorical A_j, we assume $\mathbf{dom}(A_j)$ is a finite set of category labels. We do not impose distributional assumptions on attribute values; whenever useful, we explicitly state boundedness assumptions (e.g., a bounded categorical universe or tracked numeric min/max values). A *relation* (snapshot) is a finite set $R \subseteq \prod_{j=1}^{d} \mathbf{dom}(A_j)$, with $|R| = n$ tuples. We write a tuple $t \in R$ as $t = (t_1, \ldots, t_d)$ where $t_j \in \mathbf{dom}(A_j)$ is the value of attribute A_j in t. Denote by $N \subseteq \{1, \ldots, d\}$ the indices of numerical attributes and $C = \{1, \ldots, d\} \setminus N$ the categorical attribute indices.

To model streaming scenarios we treat the data as an append-only *tuple stream* $S = \langle t^{(1)}, t^{(2)}, t^{(3)}, \ldots \rangle$, where each $t^{(i)} \in \prod_{j=1}^{D} \mathbf{dom}(A_j)$. Batch evaluation is the special case in which we treat a finite prefix (or an entire finite stream) as a relation.

[1]The name Pacha comes from Andean culture and carries complex significance, representing the earth, the cosmos, as well as space and time.

Multi-relation (join) view. Our model is intentionally relational. When data naturally spans multiple relations $R_1, \ldots, R_m$, we view queries as defined over the conceptual join $J = R_1 \bowtie R_2 \bowtie \cdots \bowtie R_m$. We note that we do *not* require materializing J; instead, algorithms operate on the base relations or on streams of joined tuples produced by a join operator (materialized or streaming). This keeps the framework applicable to both single- and multi-table workloads.

EXAMPLE 1 (Purchases relation). *Consider an e-commerce platform that records users' purchases in a relation Purchases(*region, category, total_spent, date*) where:*

$$\begin{aligned} \textbf{\textit{dom}}(\textit{region}) &= \{\textit{Africa, America, Asia, Europe}\}, \\ \textbf{\textit{dom}}(\textit{category}) &= \{\textit{Home, Kitchen, Sports, Toys}\}, \\ \textbf{\textit{dom}}(\textit{total_spent}) &= \mathbb{R}_{\geq 0}, \\ \textbf{\textit{dom}}(\textit{date}) &= \{1990\text{-}01\text{-}01, \ldots, 2025\text{-}06\text{-}30\}. \end{aligned}$$

Each tuple in Purchases corresponds to one purchase event. Note, here region *and* category *are categorical attributes, while* total_spent *and* date *are numerical (or ordered) attributes.*

B. Query Model

We consider *multidimensional aggregate queries*, which consist of (i) a collection of attribute-wise predicates that define a selection region in the data space, and (ii) an aggregate operator evaluated over the tuples in that region. Formally, a query is defined as a tuple $Q = \big(I, \{P_j\}_{j \in I}, \mathsf{AGG}\big)$, where:

- $I \subseteq \{1, \ldots, d\}$ is the set of attribute indices referenced by the selection predicates (attributes not in I are unconstrained).
- For each $j \in I$, P_j is a predicate on $\mathbf{dom}(A_j)$, which is either:
  - a *numeric range* predicate $P_j(x) : \ell_j \leq x \leq u_j$ for $j \in N$, where $\ell_j, u_j \in \mathbb{R}$; or
  - a *categorical membership* predicate $P_j(x) : x \in V_j$ for $j \in C$, where $V_j \subseteq \mathbf{dom}(A_j)$ is a finite subset. (Point/equality predicates correspond to $V_j = \{v\}$.)
- AGG is an aggregate operator. We primarily consider the common aggregates count, sum, and avg.

A tuple $t = (t_1, \ldots, t_d)$ *satisfies* query Q if $P_j(t_j)$ holds for all $j \in I$. We denote this by the indicator $\mathbb{I}[Q(t)]$. Given a finite relation R, the result of query Q is defined as

$$\mathsf{AGG}_R(Q) \;=\; \mathsf{AGG}\big(\{\, t \in R \mid \mathbb{I}[Q(t)] = 1 \,\}\big),$$

where the aggregate operator is interpreted as follows: count returns the number of tuples satisfying Q; $\mathsf{sum}(A_k)$ returns the sum of attribute A_k over the satisfying tuples, where $k \in N$ is fixed by the aggregate; and $\mathsf{avg}(A_k)$ returns the ratio of $\mathsf{sum}(A_k)$ to count over the satisfying tuples.

Disjunctions and complex predicates. Predicates that include disjunctions (for example, $(A_1 \in V_1) \vee (A_2 \in V_2)$) can be rewritten as a finite union of conjunctive predicate regions. Evaluation then reduces to computing aggregates on each conjunctive region and combining them using the inclusion-exclusion principle to avoid double-counting. Because this composition relies only on additive/decomposable properties of the underlying AGG (e.g., linearity for sum and count), our framework and algorithms naturally extend to such cases.

Wildcards and point filters. An attribute not listed in I is treated as unconstrained (a wildcard). A categorical point filter $(A_j = v)$ is the same as $V_j = \{v\}$; a numeric point filter $(A_j = x_0)$ is the degenerate interval $\ell_j = u_j = x_0$.

EXAMPLE 2 (Multidimensional aggregate query). *Consider the query that counts all purchases made in Asia in 2023. In our formal model, this query is represented as:*

$$\begin{aligned} Q_1 = \big(I = \{\textit{region}, \textit{date}\},\, & P_{\textit{region}}(x) : x \in \{\textit{Asia}\}, \\ & P_{\textit{date}}(x) : 2023\text{-}01\text{-}01 \leq x \leq 2023\text{-}12\text{-}31, \\ & \mathsf{count}\big). \end{aligned}$$

III. Organizing Multidimensional Space

In this section, we describe our framework for systematically organizing multidimensional data into a hierarchical set of "summary" regions that enable efficient aggregation. We begin with a primer on Dyadic Ranges (Section III-A) that can efficiently support 1-D range count queries and show how they can be extended to any base b (Section III-B). We then discuss how b-adic range covers generalize naturally to higher-dimensional $\mathbb{B}$-adic hypercubes (Section III-C). We then explain how to integrate categorical attributes via an AD-tree hierarchy and combine these with the numerical covers to handle mixed-type predicates (Section III-D). Our multidimensional space organization allows us to decompose arbitrary aggregate queries into a minimal collection of summary regions, laying the groundwork for the PACHA SKETCHES.

A. Primer on Dyadic Range Cover

Dyadic Ranges A *Dyadic Range* [33] is a class of intervals with some useful mathematical properties. Dyadic ranges hierarchically partition a numerical domain into intervals that grow exponentially. Every dyadic range has a power-of-two length $\ell = 2^k$, and its start index is 1 (mod ℓ) (assuming 1-indexing). That is, each point in $1 \ldots n$ is a member of exactly one dyadic range of each length $2^0, 2^1, \ldots, 2^{\log n}$. Any range within $0, \ldots, n$ has a unique, non-overlapping, minimal covering consisting of at most $2\lceil \log(n) \rceil - 3$ dyadic ranges.

Approximate Count Queries with Dyadic Ranges A standard approach to support approximate range queries is to maintain multiple sketches over dyadic ranges. In particular, one can maintain $\log(n)$ sketches corresponding to dyadic range lengths $2^0, 2^1, \ldots, 2^{\log(n)}$ [11], [30]. The first sketch summarizes individual elements, the second summarizes pairs of adjacent elements by aggregating their updates, and so on. Using this hierarchy, an arbitrary range query can be decomposed into a small number of dyadic intervals and answered by combining the corresponding point queries, requiring at most $2\lceil \log(n) \rceil - 3$ point queries in total.

An alternative line of work avoids maintaining multiple sketches by using fast range-summable hash functions. In particular, Rusu et al. [18], [19], [34] show how to answer range queries directly using AMS sketches [35], trading additional hashing complexity for reduced sketch maintenance.

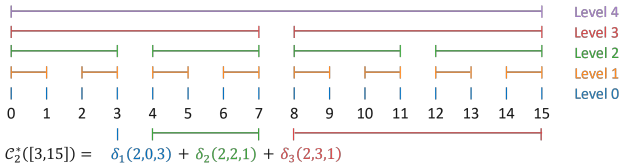


Fig. 1: b-adic range cover ($b = 2$).

Algorithm 1 Minimal b-adic range cover.

Require: $q = [\alpha, \beta]$, b
Ensure: $\mathcal{C}_b^*(q)$
1: $\mathcal{C}_b^*(q) \leftarrow \emptyset$; $l \leftarrow 0$ ▷ *Start from the first level*
2: **while** $\alpha \leq \beta$ **do**
3: $low_limit \leftarrow \left\lfloor \alpha / b^{l+1} \right\rfloor \cdot b^{l+1}$ ▷ *Check if a range from next level starts exactly at α*
4: **if** $low_limit \neq \alpha$ **then**
5: $low_limit \leftarrow low_limit + b^{l+1}$ ▷ *Add ranges of level l until reaching low_limit*
6: **while** $low_limit \neq \alpha \wedge \alpha \leq \beta$ **do**
7: $i \leftarrow \left\lfloor \alpha / b^l \right\rfloor$
8: $\mathcal{C}_b^*(q) \leftarrow \mathcal{C}_b^*(q) \cup \delta(b, l, i)$
9: $\alpha \leftarrow \alpha + b^l$
10: $high_limit \leftarrow \left(\left\lfloor \beta / b^{l+1} \right\rfloor + 1 \right) \cdot b^{l+1} - 1$ ▷ *Check if a range from next level ends exactly at β*
11: **if** $high_limit \neq \beta$ **then**
12: $high_limit \leftarrow high_limit - b^{l+1}$ ▷ *Add ranges of level l until reaching high_limit*
13: **while** $high_limit \neq \beta \wedge \alpha \leq \beta$ **do**
14: $i \leftarrow \left\lfloor \beta / b^l \right\rfloor$
15: $\mathcal{C}_b^*(q) \leftarrow \mathcal{C}_b^*(q) \cup \delta(b, l, i)$
16: $\beta \leftarrow \beta - b^l$
17: $l \leftarrow l + 1$

B. One Dimensional Range Cover

b-Adic Range Cover Hierarchical range decompositions can be constructed with different branching factors [36], [37]. b-adic ranges generalize dyadic ranges to any base b. A larger base allows coarser partitions at higher levels, reducing the total number of levels needed to cover large domains. This is especially important when the domain size is large, as smaller bases, such as 2, require deeper hierarchies, increasing both storage and query overhead. Figure 1 illustrates how b-adic ranges organize the numerical domain using the base $b = 2$. The hierarchy is organized into levels, which determine the sizes of the ranges, e.g., the ranges in level ℓ have size b^ℓ. These ranges are then indexed by i, which determines their position within that level. Based on this notation we formalize a b-adic range as $\delta(b, \ell, i) = [i \cdot b^\ell, (i+1) \cdot b^\ell)$, where b is the base, ℓ is the level, and i is the index.

Querying Using b-adic Ranges Let us now assume a range query $q = [\alpha, \beta]$. We define $\mathcal{C}_b(q)$ as the b-adic range cover of $[\alpha, \beta]$ using the base b. A b-adic range cover is composed by a set of ranges δ that cover the query range $[\alpha, \beta]$: $\bigcup_{\delta \in \mathcal{C}_b([\alpha,\beta])} \delta = [\alpha, \beta]$.
Then we can use such cover to answer an aggregate query over the range in q, i.e., $\mathsf{AGG}(q) = \mathrm{AGG}\Big(\{\, \mathrm{agg}(\delta) \mid \delta \in \mathcal{C}_b(q) \,\}\Big)$, where $\mathrm{agg}(\delta)$ is the local aggregate of the items in the b-adic range δ.

Minimal Cover Intuitively, for optimal performance, we want to minimize the number of ranges in the b-adic range cover. We say a b-adic range cover $\mathcal{C}_b^*(q)$ is *minimal* if it is the smallest set of b-adic ranges that cover the query range $[\alpha, \beta]$.

Algorithm 1 shows how to find the minimal b-adic range cover for a given range query. The algorithm works by iteratively on every level checking if the current range $[\alpha, \beta]$ is aligned with a b-adic range of the next level $\ell + 1$ (lines 3 and 10) and if not it adds as many b-adic ranges of level ℓ to the cover until the query ranges are aligned (lines 8 and 15). The algorithm terminates when the entire query range is covered, i.e., when $\alpha > \beta$.

Example 1-D Range Query To better illustrate the above algorithm, let us consider the example query $q = [3, 15]$ shown in Figure 1. We start with the lowest level $\ell = 0$ and check if the query range aligns with a b-adic range of the next level $\ell + 1 = 1$. This is not the case as the second b-adic range of level one contains $\alpha = 3$. Then we add as many b-adic ranges of level $\ell = 0$ until we reach the start of the next b-adic range at index 4. In this case the only b-adic range to add is $\delta(2, 0, 3) = [3, 4)$, and we update $\alpha = 4$. We now continue from the end of the query range ($\beta = 15$). Since a b-adic range of level one ends at 15, we treat this end as aligned. In the second iteration ($\ell = 1$), the range aligns at both ends with the b-adic range of level two, so we proceed to the next level $\ell = 2$. Now α is again not aligned with the next level, so we add the b-adic range $\delta(2, 2, 1) = [4, 8)$ and update $\alpha = 8$. $\beta = 15$ aligns with the next level, so we proceed to the last iteration ($\ell = 3$). Here the range $[\alpha, \beta] = [8, 15]$ is perfectly aligned with the b-adic range of level three, so we add the last b-adic range $\delta(2, 3, 1) = [8, 16)$ and finish the algorithm. The final result is $\mathcal{C}_2^*([3, 15]) = \{\delta(2, 0, 3), \delta(2, 2, 1), \delta(2, 3, 1)\}$.

Complexity Analysis The following theorem captures both the best-case and worst-case bounds on the number of intervals produced by the greedy algorithm.

THEOREM 1 (Size of Minimal b-adic Range Cover). *Let $q = [\alpha, \beta]$ be a one-dimensional range query over a numerical domain, and let $\mathcal{C}_b^*(q)$ denote its minimal b-adic cover. Define the range width as $n = \beta - \alpha + 1$. Then: In the best case (when q aligns perfectly with a b-adic interval), we have $|\mathcal{C}_b^*(q)| = 1$; and in the worst case, the number of intervals in the cover satisfies $|\mathcal{C}_b^*(q)| \leq 2(b-1) \cdot (2\lfloor \log_b(n) \rfloor + 1)$. Consequently, for any $b \geq 2$, the size of the cover is bounded as $|\mathcal{C}_b^*(q)| = O(b \cdot \log_b(n))$. Moreover, since b is typically a small constant (e.g., $b = 2, 4, 8$), the cover size grows logarithmically in the query width n, i.e., $|\mathcal{C}_b^*(q)| = O(\log n)$.*

Proof. We analyze the number of intervals produced by the greedy algorithm that constructs the minimal b-adic cover $\mathcal{C}_b^*(q)$ for a range $q = [\alpha, \beta]$ of width $n = \beta - \alpha + 1$.
Best case. If the range $[\alpha, \beta]$ is perfectly aligned with a b-adic interval at some level ℓ, then the algorithm selects this interval directly, and no further decomposition is required. Hence, $|\mathcal{C}_b^*(q)| = 1$.
Worst case. In the worst case, the range $[\alpha, \beta]$ is not aligned with any b-adic interval, and the algorithm must decompose it across multiple levels. Let $k = \lfloor \log_b(n) \rfloor$, where $n = \beta - \alpha + 1$. The largest b-adic interval that can fit within the query range has level at most k. The greedy algorithm constructs the cover by progressively covering the query from both endpoints,

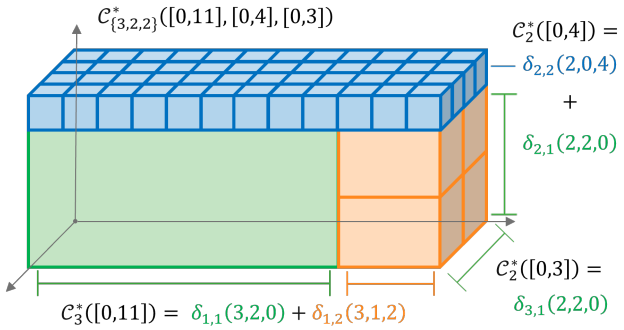


Fig. 2: $\mathbb{B}$-adic space cover ($\mathbb{B} = \{3, 2, 2\}$)

selecting the largest aligned b-adic intervals that fit in the remaining uncovered range.

At each level ℓ, at most $b-1$ intervals of size b^ℓ are needed to align the left boundary with the next level-$\ell+1$ interval. The same bound applies to the right boundary. Consequently, at most $2(b-1)$ intervals are added per level. Since there are at most $k+1$ relevant levels, the total number of intervals satisfies

$$|\mathcal{C}^*_b(q)| \leq 2(b-1)(k+1).$$

Because $k \geq 0$, we have $k+1 \leq 2k+1$. Substituting $k = \lfloor \log_b(n) \rfloor$ therefore yields

$$|\mathcal{C}^*_b(q)| \leq 2(b-1) \cdot (2\lfloor \log_b(n) \rfloor + 1).$$

Asymptotic bound. Since the number of levels is $O(\log_b n)$ and at each level we add at most $O(b)$ intervals, it follows that:

$$|\mathcal{C}^*_b(q)| = O(b \cdot \log_b n).$$

For constant b, this simplifies to $|\mathcal{C}^*_b(q)| = O(\log n)$. □

Working with Floating Point and Ordinal Data Algorithm 1 can be adapted to floating point numbers by allowing negative exponents, such that the lowest level is negative instead of zero. The only required preprocessing step is to round the floating-point range to the smallest available b-adic range, ensuring that the query range aligns with the available b-adic ranges. Finally, we can also adapt this approach to ordinal attributes by mapping them to a numerical domain, e.g., number of days since the earliest date.

C. Multidimensional Spatial Covers

$\mathbb{B}$-Adic Spatial Cover We now consider a multidimensional range query over D dimensions, defined as $Q = \{[\alpha_d, \beta_d] \mid d \in [1, D]\}$. Let $\mathbb{B} = \{b_d \mid d \in [1, D]\}$ denote the set of bases used to construct a b_d-adic range cover for each dimension d. We define a $\mathbb{B}$-adic hypercube $\Delta(\mathbb{B}, \ell, I)$ as a region in D-dimensional space formed by the Cartesian product of b_d-adic ranges $\delta(b_d, \ell, i_d)$ across all dimensions, where all ranges share the same level ℓ. To illustrate, consider the green volume in Figure 2, which corresponds to the $\mathbb{B}$-adic hypercube $\Delta(\{2,2,2\}, 2, (0,0,0))$. This hypercube is defined by the ranges $\delta_1(2,2,0) = [0,4)$, $\delta_2(2,2,0) = [0,4)$, and $\delta_3(2,2,0) = [0,4)$ along the three dimensions, respectively.

Algorithm 2 Minimal spatial $\mathbb{B}$-adic range cover.

Require: $Q = \{[\alpha_d, \beta_d] \mid d \in [1, D]\}$, $\mathbb{B} = \{b_d \mid d \in [1, D]\}$
Ensure: $\mathcal{C}^*_{\mathbb{B}}(Q)$
1: $C \leftarrow \emptyset$
2: **for all** $q \in Q$ **do**
3: $C \leftarrow C \cup \{\mathcal{C}^*_{b_d}(q)\}$ ▷ *Compute the minimal b-adic range cover for each dimension*
4: $R \leftarrow \prod_{C_d \in C} C_d$ ▷ *Build the Cartesian product of all b-adic range covers*
5: $\mathcal{C}^*_{\mathbb{B}}(Q) \leftarrow \emptyset$
6: **for all** $R_j \in R$ **do**
7: $l_{min} \leftarrow \min_{\delta(b,l,i) \in R_j} l$ ▷ *Find the smallest level in the region*
8: $R'_j \leftarrow \emptyset$
9: **for all** $\delta(b, l, i) \in R_j$ **do**
10: $R'_j \leftarrow R'_j \cup \{\text{DOWNGRADE}(\delta(b, l, i), l_{min})\}$ ▷ *Downgrade all ranges to the same level l_{min}*
11: $R_{min} \leftarrow \prod_{r \in R'_j} r$ ▷ *Build the Cartesian product of all downgraded ranges*
12: **for all** $R'' \in R_{min}$ **do**
13: $I \leftarrow \{i \mid \delta(b, l, i) \in R''\}$
14: $\mathcal{C}^*_{\mathbb{B}}(Q) \leftarrow \mathcal{C}^*_{\mathbb{B}}(Q) \cup \Delta(B, l_{min}, I)$

Using this notation, we define the $\mathbb{B}$-adic spatial cover $\mathcal{C}_{\mathbb{B}}(Q)$ as the set of $\mathbb{B}$-adic hypercubes that together fully cover the query region Q, i.e., $\bigcup_{\Delta \in \mathcal{C}_{\mathbb{B}}(Q)} \Delta = Q$.

Minimal Spatial Cover We denote $\mathcal{C}^*_{\mathbb{B}}(Q)$ as the minimal $\mathbb{B}$-adic spatial cover of Q with the smallest number of $\mathbb{B}$-adic hypercubes to cover the query space. Algorithm 2 computes this spatial cover for a given multidimensional range query Q and a set of bases $\mathbb{B}$. The algorithm works by first computing the minimal b_d-adic range cover for each dimension $d \in [1, D]$ (line 3) and then building the Cartesian product of all b-adic range covers (line 4) to map the regions contained in the query space. In the next step, the algorithm iterates over all regions. For each region it finds the smallest level ℓmin of the limiting b-adic ranges (line 7) and then downgrades all ranges to the same level ℓ_{min} by converting higher level b-adic ranges to a set of b-adic ranges of level ℓ_{min} (line 10). Finally, the algorithm constructs a new Cartesian product to map regions of the same level (line 11); for each resulting tuple of ranges, it adds a corresponding $\mathbb{B}$-adic hypercube $\Delta(\mathbb{B}, \ell_{min}, I)$ to the minimal spatial cover (line 14).

Example Figure 2 illustrates the $\mathbb{B}$-adic space cover algorithm. The query is $Q = \{[0,11], [0,4], [0,3]\}$, and we consider a $\mathbb{B}$-adic space with bases $\mathbb{B} = 3, 2, 2$. We first compute the b-adic range covers for each dimension and then take their Cartesian product, yielding four regions. Consider the first region delimited by $(\delta_{1,1}(3,2,0), \delta_{2,1}(2,2,0), \delta_{3,1}(2,2,0))$, enclosing $[0,8], [0,3], [0,3]$. Since all ranges are at level $\ell = 2$, we directly add the $\mathbb{B}$-adic cube $\Delta(\mathbb{B}, 2, (0,0,0))$ to the minimal space cover (green cube in Figure 2).

Next, consider the region delimited by $(\delta_{1,2}(3,1,2), \delta_{2,1}(2,2,0), \delta_{3,1}(2,2,0))$, enclosing $[9,11], [0,3], [0,3]$. Here, the first dimension is at level $\ell = 1$, while the others are at level $\ell = 2$. We therefore downgrade $\delta_{2,1}(2,2,0)$ and $\delta_{3,1}(2,2,0)$ to level 1, yielding four regions and four $\mathbb{B}$-adic hypercubes (orange cubes in Figure 2). Repeating this process for all regions, and filling the remaining space at level 0 (blue cubes), the minimal $\mathbb{B}$-adic spatial cover contains 53 hypercubes instead of the

240 point queries contained in Q.

THEOREM 2 (Size of $\mathbb{B}$-adic Spatial Cover under Bounded Level Spread). *Let $Q = \{[\alpha_d, \beta_d] \mid d \in [1, D]\}$ be a D-dimensional range query with widths $n_d = \beta_d - \alpha_d$, and let $\mathcal{C}^*_{\mathbb{B}}(Q)$ denote the minimal $\mathbb{B}$-adic spatial cover.*

Assume that for every region r considered during the construction, the levels $\ell_d^{(r)}$ of the corresponding b_d-adic intervals satisfy a bounded spread condition:

$$\max_d \ell_d^{(r)} - \min_d \ell_d^{(r)} \leq \kappa$$

for some constant $\kappa \geq 0$.

Then the size of the cover satisfies:

$$|\mathcal{C}^*_{\mathbb{B}}(Q)| = O\Big(\prod_{d=1}^{D} (b_d \log n_d) \cdot (\prod_{d=1}^{D} b_d)^{\kappa}\Big).$$

In particular:

- ***Best case:** If $\kappa = 0$, then*
$$|\mathcal{C}^*_{\mathbb{B}}(Q)| = O\Big(\prod_{d=1}^{D} (b_d \log n_d)\Big).$$
- ***Worst case:** If no bound on the level spread is assumed, then*
$$|\mathcal{C}^*_{\mathbb{B}}(Q)| = O\Big(\prod_{d=1}^{D} n_d\Big).$$

Proof. The algorithm first computes the 1D b_d-adic covers for each dimension. By Theorem 1, each dimension contributes at most $O(b_d \log n_d)$ intervals. Taking the Cartesian product yields at most $\prod_{d=1}^{D} O(b_d \log n_d)$ regions.

Consider a region r formed by selecting one interval per dimension, and let $\ell_d^{(r)}$ denote the level of the interval in dimension d. Let $\ell_{\min}^{(r)} = \min_d \ell_d^{(r)}$. To align all dimensions to level $\ell_{\min}^{(r)}$, each interval at level $\ell_d^{(r)}$ is decomposed into $b_d^{\ell_d^{(r)} - \ell_{\min}^{(r)}}$ intervals. Therefore, the number of resulting subregions for region r is:

$$\prod_{d=1}^{D} b_d^{\ell_d^{(r)} - \ell_{\min}^{(r)}}.$$

Under the bounded level spread assumption, we have:

$$\ell_d^{(r)} - \ell_{\min}^{(r)} \leq \kappa \quad \text{for all } d,$$

which implies:

$$\prod_{d=1}^{D} b_d^{\ell_d^{(r)} - \ell_{\min}^{(r)}} \leq \prod_{d=1}^{D} b_d^{\kappa} = (\prod_{d=1}^{D} b_d)^{\kappa}.$$

Thus, each region contributes at most $(\prod_{d=1}^{D} b_d)^{\kappa}$ subregions. Multiplying by the number of outer regions yields:

$$|\mathcal{C}^*_{\mathbb{B}}(Q)| = O\Big(\prod_{d=1}^{D} (b_d \log n_d) \cdot (\prod_{d=1}^{D} b_d)^{\kappa}\Big).$$

If $\kappa = 0$, then all levels are equal and no downgrading is required, yielding the best-case bound.

If no bound on the level spread is assumed, then in the worst case, one dimension has level 0, and all others have maximal levels $L_d = \lfloor \log_{b_d}(n_d) \rfloor$. Then:

$$\prod_{d=1}^{D} b_d^{\ell_d^{(r)} - \ell_{\min}^{(r)}} = \prod_{d=1}^{D-1} b_d^{L_d} = \prod_{d=1}^{D-1} n_d,$$

which yields the worst-case bound, which is not better than the naïve baseline, which covers the space using point queries. □

Discussion. The size of the spatial cover is determined by the spread between the levels $\ell_d^{(r)}$ across dimensions. When the levels are similar, the differences $\ell_d^{(r)} - \ell_{\min}^{(r)}$ remain small, resulting in few downgraded combinations. On the contrary, when the levels differ significantly (e.g., short ranges in some dimensions and long ranges in others), the number of combinations can grow rapidly, approaching the worst-case bound.

Practical Mitigations. In practice, the worst-case behavior can be mitigated by aligning the level distributions across the ranges used to answer a query. We propose three techniques to reduce the spread between levels across dimensions, thereby pushing the cover size closer to the best-case behavior:

- **Domain-specific bases:** Choosing dimension-specific bases b_d that match the scale of each attribute's domain helps balance the levels across dimensions. Larger domains can use larger bases, while smaller domains use smaller bases, reducing the likelihood of large level differences. This approach tailors the hierarchical decomposition to each dimension, yielding more efficient spatial covers, particularly when prior knowledge of the data distribution and query patterns is available. Consider a two-dimensional query over attributes `age` and `income`. Suppose `age` has domain size $n_1 = 100$ and `income` has domain size $n_2 = 10^6$. If both dimensions use the same base $b_1 = b_2 = 2$, then typical query ranges (e.g., $[20, 40]$ for age and $[30\text{k}, 60\text{k}]$ for income) will correspond to very different levels, e.g., $\ell_1 \approx 5$ and $\ell_2 \approx 15$, yielding a level spread of $\kappa \approx 10$. This leads to a large number of downgraded combinations. Instead, by choosing domain-specific bases such as $b_1 = 2$ for `age` and $b_2 = 10$ for `income`, the same query ranges may correspond to levels $\ell_1 \approx 5$ and $\ell_2 \approx 6$, reducing the spread to $\kappa \approx 1$. Under the bounded level spread assumption, this significantly reduces the number of combinations from exponential in κ to near-best-case behavior.
- **Query coarsening:** When approximate answers are acceptable, query ranges can be coarsened to align with higher levels in the hierarchy. Consider the example in Fig. 2, where we require 53 cubes to cover the original query region. By coarsening the query range in the second dimension from $[0, 4]$ to $[0, 5]$, we can align $\delta_{2,2}$ to level $\ell = 1$, which reduces the number of required cubes to 11. More generally, coarsening a query range to align with the hierarchy reduces the level spread κ across dimensions, which in turn reduces the number of downgraded combinations. This technique is particularly effective when queries are close to alignment with the hierarchy. Moreover, since multidimensional data is often sparse, coarsening may introduce only a limited number of additional data points, thereby improving efficiency without significantly sacrificing accuracy.

- **Projected covers:** For workloads where some dimensions are frequently omitted, it is beneficial to maintain projected covers for common subsets of dimensions, which we refer to as subspaces. This avoids unnecessary expansion in unused dimensions and reduces the number of combinations introduced during downgrading. From the perspective of level alignment, including unused dimensions can artificially increase the level spread $\kappa = \max_d \ell_d - \min_d \ell_d$. In particular, dimensions that are not constrained by the query typically span the full domain and therefore correspond to higher levels, while other queried dimensions may require only lower levels. This increases κ and leads to a large number of downgraded combinations. For example, instead of maintaining a single mapping for queries over (d_1, d_2, d_3), one can maintain separate mappings for (d_1, d_2) and (d_1, d_2, d_3). A query involving only d_1 and d_2 can then be evaluated using the projected subspace cover, avoiding the inclusion of d_3, which would otherwise increase the level spread and degrade performance. This approach is particularly effective when query workloads are predictable from domain knowledge or from semantic relationships among attributes. Additionally, prior work suggests that real-world analyses often limit queries to a small number of dimensions (e.g., up to four) when reconstructing probability distributions [38], [39], so not all combinations need to be retained.

D. Mapping Mixed-Type Attributes

With Algorithm 2, we have a solid foundation to improve the performance of multidimensional range queries. However, critical factors remain to avoid the worst case and improve the average case for practical applications. Next, we discuss how to map multidimensional data to exploit the $\mathbb{B}$-adic spatial cover efficiently.

Data Types and Their Predicates. Unlike related works [40], we do not assume all dimensions are cardinal. Most real-world datasets mix categorical and numerical data, which are filtered with different predicates. Categorical attributes typically use equality predicates, $t_d = c$, while numerical attributes use range predicates, $t_d \in [\alpha, \beta]$. Naively applying Algorithm 2 to categorical data would produce a single level per attribute, forcing all equality predicates to use $\mathbb{B}$-adic hypercubes at level $\ell = 0$, leading to the worst case scenario.

Therefore, we propose mapping multidimensional data into two components. Let the incoming tuple $t = (t_1, \ldots, t_D)$ be a D-dimensional vector, where t_d is the value of the attribute d. First, we split the tuple into two parts: t_{cat} and t_{num}, where t_{cat} contains the categorical attributes and t_{num} contains the numerical attributes. We now proceed to map each part separately at different generalization levels. To illustrate this process, let us assume that the tuple $t = (t_1, t_2, t_3, t_4, t_5, t_6)$ contains three categorical attributes $t_{cat} = (t_1, t_2, t_3)$ and three numerical attributes $t_{num} = (t_4, t_5, t_6)$. We can map the tuple t as follows:

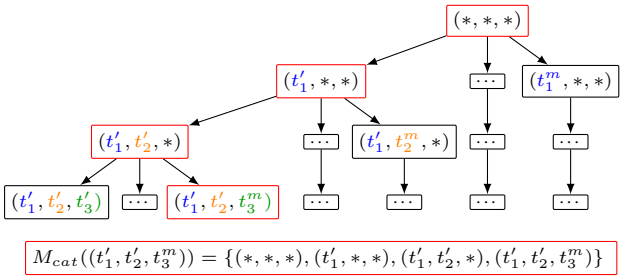


Fig. 3: Example AD-Tree for three dimensions. Red nodes represent the encodings of the categorical attributes for the tuple $t_{cat} = (t'_1, t'_2, t^m_3)$.

Mapping Categorical Attributes. Multidimensional hierarchical representations based on combinations of per-dimension generalizations have been studied previously, e.g., in the context of hierarchical heavy hitters [41]. To map the categorical attributes $M_{cat}(t_{cat})$, we conceptually rely on the All-Dimensions tree (AD-tree) [31], but we do not materialize the tree or any of its nodes. The AD-tree defines a hierarchy over all possible combinations of categorical attributes, resulting in an exponential number of nodes if fully materialized.

Instead, we use this structure purely to define a deterministic encoding of categorical attributes. Given a fixed ordering of the categorical dimensions, each tuple t_{cat} is mapped to $|t_{cat}| + 1$ identifiers corresponding to progressively more general representations, obtained by replacing suffixes of the tuple with wildcards. For our example in Figure 3 $t_{cat} = (t'_1, t'_2, t^m_3)$, we define: $M_{cat}(t_{cat}) = \{(*, *, *), (t'_1, *, *),$-$(t'_1, t'_2, *), (t'_1, t'_1, t^m_3)\}$.

These identifiers are not stored in a tree structure; instead, they serve as keys in our sketch-based data structures, which maintain aggregates at different levels of generalization. When a query is issued, we determine which identifiers satisfy the query predicates and retrieve the corresponding aggregates.

For example, a query with predicates $t_1 = t'_1$ and $t_2 = t'_2$ matches a single identifier, namely $(t'_1, t'_2, *)$. In contrast, a query with only $t_2 = t'_2$ matches multiple identifiers, specifically $\{(t^i_1, t'_2, *) \mid \forall t^i_1 \in \textbf{dom}(t_1)\}$, whose number grows with $|\textbf{dom}(t_1)|$.

THEOREM 3 (Number of categorical identifiers matched by a query). *Let $I \subseteq [1, D]$ denote the set of categorical dimensions constrained by equality predicates in Q_{cat}, and let I_t be the prefix of the dimension ordering up to the last constrained dimension. Then, the number of identifiers matched by the query is*

$$|Q'_{\mathsf{cat}}| = \frac{\prod_{d \in I_t} m_d}{\prod_{d \in I} m_d},$$

where $m_d = |\textbf{dom}(t_d)|$ is the domain size of dimension d.

Proof. Recall that identifiers are constructed by progressively instantiating dimensions in a fixed order. A query matches all identifiers that agree with its equality predicates.

For each dimension $d \in I_t$, there are two cases. If $d \in I$, the query specifies a value, and thus only one value is consistent with the query. If $d \notin I$, the dimension is unconstrained, and

all m_d values are possible. Therefore, the total number of matching identifiers is given by the product over all dimensions in I_t, where each constrained dimension contributes a factor of 1 and each unconstrained dimension contributes a factor of m_d:

$$|Q'_{\mathsf{cat}}| = \prod_{d \in I_t \setminus I} m_d.$$

Rewriting this expression yields the stated form:

$$|Q'_{\mathsf{cat}}| = \frac{\prod_{d \in I_t} m_d}{\prod_{d \in I} m_d}.$$

For comparison, a flat enumeration without the AD-tree would require $m^{D-|I|}$ lookups. □

This behavior highlights the importance of the dimension ordering, which defines the hierarchy of generalizations and directly affects the query-answering cost. Queries that align with earlier dimensions in the ordering can be answered using fewer, more specific identifiers, whereas queries on later dimensions require aggregating over a larger number of identifiers. If workload information is available, placing frequently queried attributes earlier in the ordering improves efficiency. Otherwise, standard heuristics such as ordering by attribute cardinality, skew, or semantic relationships can be used.

Mapping Numerical Attributes. Now, we proceed mapping the numerical attributes $M_{num}(t_{num})$. We start by mapping t_{num} to a $\mathbb{B}$-adic hypercube of the lowest level $\Delta_0(\mathbb{B}, 0, I_0)$. Note that the $\mathbb{B}$-adic multidimensional space consists only of the dimensions containing numerical attributes (t_4, t_5, t_6 in our example). Now we can use Δ_0 to compute the indexing of the tuple formed by t_{num} in higher levels. This leaves us with a set of L $\mathbb{B}$-adic hypercubes, where $L-1$ is the highest level of the $\mathbb{B}$-adic space to be considered, i.e., $M_{num}(t_{num}) = \{\Delta(\mathbb{B}, \ell, I_\ell) \mid \ell \in [0, L-1]\}$.

Combining the Two Mappings Finally, we combine the two mappings by creating a Cartesian product, i.e., $M(t) = M_{cat}(t_{cat}) \times M_{num}(t_{num})$. This gives us $(|t_{cat}|+1)L$ regions for the tuple t in the multidimensional space, where the regions become general, starting from the most specific location, in our example $((t_1, t_2, t_3),\ \Delta(\mathbb{B}, 0, I_0))$, to the most general $((*, *, *), \Delta(\mathbb{B}, L-1, I_{L-1}))$.

IV. Pacha Sketches

So far, we have discussed how we can decompose multidimensional data points into a set of categorical and numerical cover regions. We now turn our attention to efficient (approximate) aggregation over these regions. Pacha Sketches realize this by combining hash-based sketches to maintain local aggregates for every discretized region in the multidimensional space, and indexing structures to efficiently prune uninhabited regions before querying the base sketches. This allows Pacha Sketches to leverage fast cover-based lookup to answer any multidimensional aggregate query. In what follows, we formally present the update and query algorithms, and finally analyze their time, space, and accuracy trade-offs.

A. Constructing Pacha Sketches

Overall Architecture Figure 4 illustrates the overall approach. At a high level, Pacha Sketches maintains:

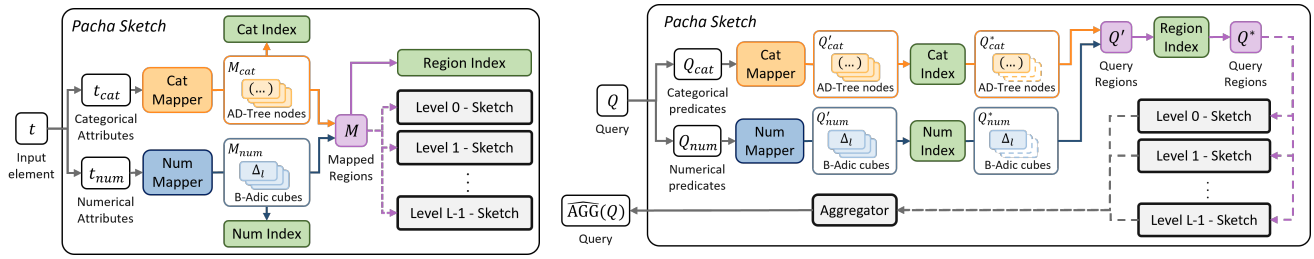
- L **base sketches** each corresponding to a different level of granularity in the $\mathbb{B}$-adic space cover.
- **Indexing structures** to efficiently prune uninhabited regions of the multidimensional space. These include a categorical index to track which combinations of categorical values are present in the data based on the AD-tree encodings, a numerical index to track which $\mathbb{B}$-adic cubes contain data points, and a region index to track which combinations of categorical and numerical regions are populated.
- **Categorical mapper** that uses an AD-tree to map the categorical attributes of every incoming tuple to a set of AD-tree nodes.
- **Numerical mapper** that uses a $\mathbb{B}$-adic space cover to locate the regions in the multidimensional space that contain the coordinates of the numerical attributes from the incoming tuple.

A key effect of the curse of dimensionality is that it increases data sparsity with increasing dimensionality. Large portions of the multidimensional space are often unpopulated, making queries over such regions not only wasteful but also error-prone—particularly for structures like Count-Min Sketches, which tend to overestimate frequencies in sparse domains [11]. Pacha Sketches are designed to efficiently answer multidimensional aggregate queries while avoiding exhaustive exploration of the space. It uses a combination of three lightweight indexing structures to prune unpopulated regions and minimize the number of sketch lookups needed, thereby mitigating the effects of dimensional sparsity.

It is worth noting that Pacha Sketches's architecture is modular: users can customize the base sketching and indexing components to suit different data or accuracy requirements, and to support new aggregates. By default, Pacha Sketches can support either count, sum, average, min, or max aggregates using variations of the Count-Min Sketch [11] as the underlying base sketches for different aggregates and Bloom Filters [32] for index pruning. We chose these structures for their compactness and well-understood error guarantees. Importantly, Pacha Sketches' accuracy, space and mergeability inherit directly from these primitives. Hence, as long as the underlying primitives are mergeable, Pacha Sketches remains mergeable when using the same configurations ($\mathbb{B}$-adic bases, categorical attribute ordering, and materialized subspaces).

To initialize Pacha Sketches, users configure: (i) the number of levels L to maintain in the space decomposition, (ii) the set of domain values for each categorical attribute and their ordering (to build AD-tree nodes), (iii) the base values $\mathbb{B}$ used for the $\mathbb{B}$-adic space cover, and (iv) the parameters of the underlying sketches and indexes.

Updating Pacha Sketches Figure 4a illustrates how to update a Pacha Sketch with a tuple $t = (t_1, \ldots, t_D)$. We first separate categorical and numerical attributes, i.e.,



(a) Update (b) Query

Fig. 4: PACHA SKETCHES illustration for answering for (a) Update: each tuple is mapped based on the categorical (orange) and numerical (blue) regions, updating the indexing structures and the base sketches at every level. (b) Query: categorical predicates map to AD-tree nodes, numerical predicates map to $\mathbb{B}$-adic cubes, and the union of these regions is queried in the base sketches.

Algorithm 3 Update Pacha sketch.

Require: PS^T, $t = (t_1, \ldots, t_d)$, C, N
Ensure: $PS^{T \cup t}$

1: $t_{cat} \leftarrow (t_i)_{i \in C}$ ▷ *Map t in the mult. space*
2: $t_{num} \leftarrow (t_j)_{j \in N}$
3: $M_{cat} \leftarrow PS.\text{MAPPINGCATEGORICAL}(t_{cat})$
4: $M_{num} \leftarrow PS.\text{MAPPINGNUMERICAL}(t_{num})$
5: $M \leftarrow M_{cat} \times M_{num}$ ▷ *Iterate over all mapped regions*
6: **for all** $(m_{cat}, m_{num}) \in M$ **do**
7: $PS.\text{UPDATECATEGORICALINDEX}(m_{cat})$ ▷ *Index regions*
8: $PS.\text{UPDATENUMERICALINDEX}(m_{num})$
9: $PS.\text{UPDATEREGIONINDEX}((m_{cat}, m_{num}))$
10: $\Delta(B, \ell, I) \leftarrow m_{num}$
11: $PS.\text{UPDATEBASESKETCH}(\Delta.\ell, (m_{cat}, m_{num}))$ ▷ *Update the base sketch in level $\Delta.\ell$*

$t_{cat} = \{t_i \mid i \in C\}$ and $t_{num} = \{t_j \mid j \in N\}$. The categorical mapper traverses the AD-tree, collecting node identifiers along the path from the root to the leaf corresponding to the combination in t_{cat} and updating the categorical index (orange path in Figure 4a). In parallel, the numerical mapper computes L $\mathbb{B}$-adic cubes containing t_{num} and updates the numerical index (blue path). It then combines categorical nodes and $\mathbb{B}$-adic cubes, as shown in Section III-D, to form regions for updating the base sketches and the region index later. Algorithm 3 details the update procedure for a PACHA SKETCH PS^T with an incoming tuple t. First (lines 1–4), the tuple's location is mapped in the multidimensional space as described in Section III-D. We then iterate over all $(|t_{cat}|+1) \cdot L$ mapped locations (line 5), updating the categorical, numerical, and region indexes (lines 7-9). Finally, we update the corresponding base sketch to reflect the region's aggregated statistics. Specifically, we update the base sketch at level $\Delta.\ell$, where $\Delta(B, \ell, I)$ is the $\mathbb{B}$-adic hypercube derived from the numerical mapping, incorporating both categorical and numerical mappings to identify the region correctly. Now we consider t to be part of $PS^{T \cup t}$.

Complexity Analysis For every incoming element t we update in PACHA SKETCH the categorical index $(|t_{cat}| + 1)$ times, the numerical index L times, the region index $(|t_{cat}| + 1) \cdot L$ times, and every base sketch $(|t_{cat}| + 1)$ times. Considering that we use Bloom Filters for the indexing structures, each time we have to update the categorical indexing $(O(k_{cat}))$, the numerical indexing $(O(k_{num}))$, and the regional indexing $(O(k_{reg}))$. Finally, we update the base sketch of the level $\Delta.\ell$ $(O(k_{cm})$ if it is a Count-Min Sketch). In general, we can denote k as the sum of hash operations used in the PACHA SKETCH, i.e., $k = k_{cat} + k_{num} + k_{reg} + k_{cm}$. Then, the complexity of updating a PACHA SKETCH with an incoming element t is: $O(|t_{cat}| \cdot L \cdot k)$.

B. Querying Pacha Sketches

Figure 4b and Algorithm 4 show how to answer a multi-dimensional aggregate query $Q = (I, \{P_j\}_{j \in I}, \mathsf{AGG})$ with a PACHA SKETCH. As in the update procedure, we first separate categorical and numerical predicates, $Q_{cat} = \{P_i \mid i \in C\}$ and $Q_{num} = \{P_j \mid j \in N\}$; if a predicate on a dimension is absent, it is taken to select all values in that dimension. The categorical mapper selects the highest AD-tree nodes satisfying Q_{cat} (line 3), as described in Section III-D, while the numerical mapper computes the minimal $\mathbb{B}$-adic space cover for Q_{num} (line 4), as described in Section III-C. Because data is typically sparse under the curse of dimensionality, many regions are uninhabited, so we use the indexing structures to prune them. First, before forming combinations of categorical nodes and $\mathbb{B}$-adic cubes, and then again after combining to remove regions that are not indexed (lines 5, 6, and 8). After pruning, we combine the relevant AD-tree nodes Q^*_{cat} with the minimal $\mathbb{B}$-adic hypercubes Q^*_{num} (line 7), refine the resulting set via the region index to obtain the final query regions Q^*, and for each (q'_{cat}, q'_{num}) in Q^* query the base sketch at level $\Delta.\ell$ where $\Delta(B, \ell, I)$ is the $\mathbb{B}$-adic hypercube from the region q'_{num}; aggregating those estimates yields the final result $\widehat{AGG}(Q)$.

Complexity Analysis. We analyze the expected complexity of Algorithm 4 by decomposing it into three parts: (1) identifying categorical regions $|Q'_{\mathsf{cat}}|$, (2) computing the numerical space cover $|Q'_{\mathsf{num}}|$, and (3) determining the number of base sketch lookups $|Q^*|$.

For the categorical space, the number of regions explored depends on the structure of the AD-tree and the query predicates. Theorem 3 provides a bound on the number of AD-tree nodes that satisfy a given query, which directly translates to $|Q'_{\mathsf{cat}}|$.

Algorithm 4 Query Pacha sketch.

Require: PS, $Q = (p_1, \ldots, p_d)$, C, N
Ensure: $\widehat{AGG}(Q)$
1: $Q_{cat} \leftarrow (p_i)_{i \in C}$
2: $Q_{num} \leftarrow (p_j)_{j \in N}$
3: $Q'_{cat} \leftarrow PS.\text{GETPRUNEDNODES}(Q_{cat})$ ▷ *Prune nodes in the categorical space*
4: $Q'_{num} \leftarrow PS.\text{GETMINIMALSPACECOVER}(Q_{num})$ ▷ *Compute the minimal $\mathbb{B}$-adic space cover*
5: $Q^*_{cat} \leftarrow PS.\text{REMOVENONINDEXED}(Q'_{cat})$ ▷ *Remove unindexed regions*
6: $Q^*_{num} \leftarrow PS.\text{REMOVENONINDEXED}(Q'_{num})$
7: $Q' \leftarrow Q^*_{cat} \times Q^*_{num}$
8: $Q^* \leftarrow PS.\text{REMOVENONINDEXED}(Q')$ ▷ *Iterate over all query regions*
9: $EST \leftarrow \emptyset$
10: **for all** $(p'_{cat}, p'_{num}) \in Q^*$ **do**
11: $\Delta(\mathbb{B}, \ell, I) \leftarrow p'_{num}$
12: $EST \leftarrow EST \cup PS.\text{QUERYBASESKETCH}(\Delta.\ell, (p'_{cat}, p'_{num}))$ ▷ *Query the base sketch in level $\Delta.\ell$*
13: $\widehat{AGG}(Q) \leftarrow PS.\text{AGGREGATEESTIMATES}(EST)$ ▷ *Aggregate estimates from all regions*

For the numerical space, we use the greedy $\mathbb{B}$-adic space cover algorithm (Algorithm 2). The expected number of resulting regions is $|Q'_{\mathsf{num}}| = |\mathcal{C}^*_{\mathbb{B}}(Q_{\mathsf{num}})|$ (see Theorem 2).

After pruning uninhabited categorical and numerical regions, we remain with $|Q^*_{\mathsf{num}}| \leq |Q'_{\mathsf{num}}|$ and $|Q^*_{\mathsf{cat}}| \leq |Q'_{\mathsf{cat}}|$ candidate regions, respectively. The cartesian product of these sets gives us the regions in the data space to query in the region index, which we define as $Q' = Q^*_{\mathsf{cat}} \times Q^*_{\mathsf{num}}$, and after pruning as Q^*, where $|Q^*| \leq |Q'|$. These regions, Q^*, are the ones we finally query in the base sketches to compute the aggregate estimate. Summing up the cost of locating all regions, pruning them, and querying the base sketches yields an overall complexity of

$$O\left(|Q'_{\mathsf{cat}}| + |Q'_{\mathsf{num}}| + |Q'|\right),$$

as $|Q^*| \leq |Q'|$.

Discussion. Now we proceed to analyze how this bound improves upon the naive approach, which requires enumerating all possible point queries in the query space, $O(|Q|)$. Q'_{cat} and Q'_{num} are deterministic and depend on the query predicates, the structure of the AD-tree, and $\mathbb{B}$-adic space cover, respectively. Both components exhibit worst-case exponential growth in the number of dimensions, which can be mitigated by exploiting the structure of the data and query predicates. Even without additional optimizations, we obtain a structural improvement by decomposing the query space into categorical and numerical components. In particular, while a naive enumeration requires exploring the full Cartesian product $|Q| = \prod_{d=1}^{D} n_d$, our approach separates this into two terms, $|Q'_{\mathsf{cat}}|$ and $|Q'_{\mathsf{num}}|$, corresponding to categorical and numerical dimensions, respectively. Since

$$\prod_{d=1}^{D} n_d \gg \prod_{d \in N} n_d + \prod_{d \in N} n_d,$$

this decomposition already avoids the full combinatorial explosion, even before considering pruning effects.

On the other hand, $|Q'|$ depends on the data distribution, as it results from pruning uninhabited regions before the building the Cartesian product. To make this bound more interpretable, let $R = |Q'_{\mathsf{cat}}| \cdot |Q'_{\mathsf{num}}|$ denote the number of regions in the hierarchical decomposition without pruning. Since Q' is the Cartesian product of the pruned categorical and numerical regions, we have $|Q'| \leq R$.

In the worst case, where the hierarchical decomposition degenerates into small regions or point queries, we have $|Q| \approx R$. However, in high-dimensional settings, data is typically sparse, and many such regions remain uninhabited, particularly at finer granularities. As a result, pruning eliminates a large fraction of candidate regions, yielding $|Q'| \ll R \approx |Q|$.

Conversely, when the space is covered by larger regions (i.e., the worst-case bounds of Theorems 3 and 2 are not met), the number of regions R is already much smaller than $|Q|$. Moreover, with pruning, this value can be further reduced $|Q'| \leq R \ll |Q|$.

Therefore, across both regimes, we expect $|Q'| \ll |Q|$ in practice, leading to a substantial improvement over the naive $O(|Q|)$ approach, even without additional heuristics for optimizing the AD-tree or the $\mathbb{B}$-adic space cover.

C. Space and Error Analysis

We designed PACHA SKETCHES to be modular, allowing users to choose the base sketches and indexing structures that best fit their needs. This is because the accuracy of any summarization structure depends strongly on data characteristics, such as the cardinality of categorical attributes and the distributions of numerical attributes.

To illustrate this, recall Example 1 and consider that the cardinality of the categorical attributes is relatively low, so the total number of distinct values is $|\mathbf{dom}(region)| \cdot |\mathbf{dom}(category)| = 20$. This implies that the corresponding materialized AD-tree can have at most 26 nodes, making a Bloom filter inefficient for categorical indexing; bitmaps suffice instead. However, for larger retail companies with hundreds of regions and categories, Bloom filters become necessary. Similarly, users may want to change the base sketches for various reasons: (i) different error guarantees for the base sketches, e.g., using Count-Sketches [42] instead to obtain unbiased frequency estimates, (ii) support for new aggregates, such as second data moments and join size estimates [18], [19], [34] using AMS sketches [35], and (iii) provide privacy guarantees using differentially private Count-Min sketches [43].

Memory usage. The space complexity of a PACHA SKETCH depends on the choice of the base sketches and indexing structures:

$$mem(PS) = mem(F_{cat}) + mem(F_{num}) + mem(F_{reg}) + L \cdot mem(BS),$$

where F_{cat}, F_{num}, and F_{reg} are the indexing structures, and BS is a base sketch.

To better illustrate this, consider a PACHA SKETCH with L levels using optimally configured Bloom filters with false positive rate p, i.e.,

$$mem(BF(n,p)) = -\frac{n \ln(p)}{8(\ln 2)^2},$$

and Count-Min sketches with error ε and δ, i.e.,

$$mem(CM(\varepsilon,\delta)) = 4 \cdot \left\lceil \frac{e}{\varepsilon} \right\rceil \cdot \left\lceil \ln\left(\frac{1}{\delta}\right) \right\rceil.$$

From Section IV-A, we know that every incoming tuple is causing $(|t_{cat}|+1)$ updates to the categorical index, L updates to the numerical index, and $(|t_{cat}|+1)\cdot L$ updates to the region index. Therefore, the total memory usage to meet the desired accuracy guarantees is:

$$mem(PS) = (|t_{cat}|L + |t_{cat}| + 2L + 1)\left(-\frac{n\ln(p)}{8(\ln 2)^2}\right) + 4L \cdot \left\lceil \frac{e}{\varepsilon} \right\rceil \cdot \left\lceil \ln\left(\frac{1}{\delta}\right) \right\rceil$$

For simplicity, we can summarize this as the following upper bound:

$$mem(PS) = O\left(L\left(n\log\left(\frac{1}{p}\right) + \frac{1}{\varepsilon}\log\left(\frac{1}{\delta}\right)\right)\right).$$

Thus, the memory usage grows linearly with the expected number of processed tuples n and the number of maintained levels L, inversely with the sketch accuracy ε, and logarithmically with the confidence parameters p and δ.

This analysis assumes that the same sketch configuration is used uniformly across all levels for simplicity. However, PACHA SKETCHES are modular by design, and its components can be adapted to the characteristics of the workload and data distribution. In particular, higher levels of the $\mathbb{B}$-adic decomposition typically aggregate over larger regions and therefore exhibit lower effective cardinality. In such cases, it may be more efficient to replace approximate sketches with exact data structures, e.g., by maintaining exact counters rather than Count-Min sketches. Similarly, if the domain of categorical attributes is small, one can replace Bloom filters with exact representations such as bitmaps.

More generally, memory can be allocated non-uniformly across levels, assigning fewer resources to higher levels where fewer distinct values are expected. These optimizations can significantly reduce memory usage in practice without affecting correctness.

Error guarantees. In general terms, the error of the PACHA SKETCHES' estimates arises from two sources: (i) approximation error in the base sketches and (ii) false positive regions that are not pruned by the indexing structures. We define the total error as

$$E_{PS} = \sum_{r\in R^*} E_{BS}(r) + \sum_{r\in Q_{FP}} \widehat{AGG}(r),$$

where R^* is the set of true positive regions containing data points, $Q_{FP} = Q^*/R^*$ is the set of false positive regions, and $E_{BS}(r)$ is the error of the base sketch for region r.

This expression is taken in expectation, since both the sketching error and the presence of false-positive regions are random variables arising from the probabilistic guarantees of the underlying data structures. For example, consider answering count queries using a PACHA SKETCH with three Bloom filters with identical false positive rate p, and a set of Count-Min sketches with error parameter ε and δ. If the PACHA SKETCH maps a query Q to R regions, then the expected estimate is

$$\widehat{c}(Q) = c(Q) + p^3 R\mu + R^*\varepsilon n,$$

where μ is the average countPl of the regions in Q_{FP} and n is the total number of processed tuples.

Since the estimation error depends on the number of queried regions, there is no non-trivial error bound independent of how the total query region is populated. Therefore, controlling the number of queried regions is critical to balancing accuracy and efficiency.

Controlling the number of regions. From the error analysis, we observe that both the expected error and its high-probability bound scale linearly with the number of regions to which a query is mapped. Therefore, reducing the number of regions not only improves query performance but also directly improves accuracy.

To this end, we incorporate the practical mitigations from Section III-C into PACHA SKETCHES. In the following, we describe how these techniques are integrated into the query processing pipeline.

Aligning numerical predicates. Even though the minimal $\mathbb{B}$-adic space cover significantly reduces the number of regions to query, its size can still grow exponentially in the worst case (Section III-C), resulting in large R and R^*. To address this, we introduce a parameter τ that bounds the maximum number of $\mathbb{B}$-adic cubes generated for a query. During query processing (Algorithm 4), instead of computing the $\mathbb{B}$-adic cover once (line 4), we invoke Algorithm 5 to ensure that $|Q'_{\mathsf{num}}| \le \tau$. Specifically, let the predicate range in dimension d be $q_d = [\alpha_d, \beta_d]$. For a given coarsening level ℓ, we align q_d to $\mathbb{B}$-adic intervals of level ℓ. Denoting the base of dimension d by b_d, the aligned interval is given by

$$q_d^{(\ell)} = \left[\left\lfloor \frac{\alpha_d}{b_d^\ell} \right\rfloor \cdot b_d^\ell,\ \left\lceil \frac{\beta_d + 1}{b_d^\ell} \right\rceil \cdot b_d^\ell - 1\right].$$

The aligned query is then $Q^{(\ell)}_{\mathsf{num}} = \{q_d^{(\ell)} \mid d \in N\}$. We iteratively increase the coarsening level ℓ and recompute the $\mathbb{B}$-adic cover until $|Q'_{\mathsf{num}}| \le \tau$. This predicate coarsening strategy complements our pruning and subspace-materialization strategies (Section III-C) by explicitly limiting the number of regions queried.

Coarsening introduces an additional source of approximation error because the aligned query Q_c may include regions outside the original query Q. Assuming that Q is mapped to a coarser query Q_c, the total estimation error arises from three sources: (i) the approximation error of the base sketches, (ii) false-positive regions that are not pruned by the indexing structures, and (iii) the coarsening error due to widening the query. Accordingly, the error analysis becomes

$$E_{PS} = \sum_{r\in R_c^*} E_{BS}(r) + \sum_{r\in Q_{cFP}} \widehat{AGG}(r) + \sum_{r\in Q_c\setminus Q} AGG(r),$$

where R_c^* is the set of true-positive regions for the coarsened query Q_c, $Q_{cFP} = Q_c^* \setminus R_c^*$ is the set of false-positive regions, and $Q_c \setminus Q$ contains the regions introduced by coarsening that are not part of the original query Q.

The parameter τ therefore controls the trade-off between accuracy and efficiency. Smaller values limit the number

Algorithm 5 Alignment of numerical predicates.

Require: Q_{num}, τ
Ensure: Q'_{num}
1: $\ell \leftarrow 0$
2: $Q'_{num} \leftarrow$ GETMINIMALSPACECOVER(Q_{num})
3: **while** $|Q'_{num}| > \tau$ **do**
4: $\quad \ell \leftarrow \ell + 1$
5: $\quad Q_{num} \leftarrow$ COARSENQUERY(Q_{num}, ℓ)
6: $\quad Q'_{num} \leftarrow$ GETMINIMALSPACECOVER(Q_{num})
7: **return** Q'_{num}

of queried regions and improve runtime, but may introduce additional approximation error due to coarser alignment. At the same time, coarsening can reduce the sketching and false-positive errors by reducing the number of queried regions. In particular, we can generally expect $|R_c^*| < |R^*|$ and $|Q_{cFP}| < |Q_{FP}|$. In practice, τ can be chosen based on resource constraints or desired latency, and we evaluate its impact in Section V-I.

Maintaining numerical subspaces. In practice, queries often involve only a small subset of numerical dimensions. Prior work shows that users typically restrict analysis to at most 3-4 dimensions when reconstructing distributions [38], [39]. In such cases, constructing $\mathbb{B}$-adic covers over the full set of numerical dimensions introduces unnecessary combinations and increases both R and R^*.

To address this, PACHA SKETCHES can maintain projected subspaces for selected subsets of numerical attributes (e.g., single dimensions and pairs). Queries are then evaluated on the smallest subspace that contains all referenced attributes, avoiding the expansion into irrelevant dimensions.

This approach is particularly beneficial when: (i) queries consistently involve low-dimensional subsets, and (ii) the full-dimensional space leads to large level spreads and cover sizes.

The selection of subspaces is currently based on workload characteristics and domain knowledge, and thus constitutes a heuristic rather than a formally optimal strategy. In practice, common choices include all single dimensions and frequently co-occurring pairs.

Maintaining additional subspaces also increases update cost, as each tuple must be inserted into multiple projections, but significantly improves query performance and accuracy by reducing the number of regions.

V. EVALUATION

We evaluated PACHA SKETCHES to investigate (a) its overall accuracy and efficiency compared to the state-of-the-art; (b) the scalability of the error guarantees; (c) its overall performance with respect to increasing number of insertions; (d) its accuracy with limited memory budget; and (e) the impact of different construction parameters on its accuracy. Overall, we observe that PACHA SKETCHES offers scalable accuracy guarantees, showing high accuracy and efficiency even for large datasets with more than 10 dimensions and millions of tuples. Additionally, PACHA SKETCHES consistently outperforms the state-of-the-art by up to three orders of magnitude in estimation accuracy and by two orders of magnitude in query performance, while using the same memory budget.

TABLE I: Comparison of Pacha and Omni Sketch.

	Pacha Sketch	**Omni Sketch**
State representation	Region aggregates	Per-dimension samples
Organization	Hierarchical regions	Independent dimensions
Estimation approach	Region aggregation	Sample intersection
Sparse-region pruning	Explicit	None
Error control	Query coarsening	Conservative fallback

A. SOTA Baseline

We compare PACHA SKETCHES against OmniSketch [30] (Table I), the current state-of-the-art sketch for multidimensional aggregate queries over data streams.

OmniSketch maintains one sketch per attribute. In our notation, for attributes $A_1, \ldots, A_D$, it constructs a collection of sketches

$$\{CM_1, CM_2, \ldots, CM_D\},$$

where each CM_d is a Count-Min-style sketch of width w and depth d. Unlike standard Count-Min sketches, each cell $CM_d[i,j]$ stores not only a counter $\mathsf{cnt}(CM_d[i,j])$, but also a bounded-size sample $S_d(i,j)$ of tuple identifiers, maintained using k-minwise hashing with a limited size B [44].

OmniSketches are originally designed for estimating count point queries $Q = (I, \{P_j\}_{j \in I}, \mathsf{count})$, where each predicate P_j is a equality predicate (e.g., $A_j = v$). OmniSketches processes each predicate P_j independently, mapping its value to sketch indices in CM_j via the corresponding hash functions, yielding a set of relevant cells. Let $C(Q)$ denote the set of all such cells across all attributes and rows. OmniSketch estimates the result by computing the intersection of the samples stored in these cells,

$$S_\cap = \bigcap_{CM_d[i,j] \in C(Q)} S_d(i,j),$$

and scaling the intersection size as

$$\widehat{count}(Q) = \frac{n_{\max}}{B} \cdot |S_\cap|,$$

where $n_{\max} = \max_{CM_d[i,j] \in C(Q)} \mathsf{cnt}(CM_d[i,j])$.

In practice, OmniSketch employs two estimation regimes depending on the size of the sample intersection. When the intersection $S_\cap$ is sufficiently large, the estimate follows from scaling the intersection size, as shown above. However, when $|S_\cap|$ is too small, the sampling-based estimate becomes unreliable. In this case, OmniSketch falls back to a conservative upper-bound estimate to preserve its theoretical error guarantees.

This behavior has important practical implications: queries with small result sizes (e.g., low-selectivity predicates) are more likely to trigger the fallback regime, leading to substantially higher estimation error. In contrast, queries with large result sizes (e.g., heavy hitters) are more likely to be estimated accurately using the scaled intersection.

Range predicates are supported using dyadic range decomposition (Section III-A), which decomposes each range into $O(\log(n_d))$ intervals. Each interval is processed as a separate query, and the final estimate is obtained by intersecting all samples.

Comparison to PACHA SKETCHES. Both OmniSketch and PACHA SKETCHES build on Count-Min-style hashing and

TABLE II: Overview of evaluation datasets. Cat. and Num. denote the number of categorical and numerical dimensions. Card. is the maximum number of categorical value combinations. Bases lists the bases for the $\mathbb{B}$-adic space, and #Subspc. is the number of materialized subspaces.

Dataset	Rows	Cat.	Num.	Card.	Max dom	Bases	#Subspc.
Lineitem (Ltm)	48M	5	5	4K	10K	5,5,5,10,2	13
Online Retail (OR)	542K	3	3	14K	170K	4,2,2	7
US Census (US)	377K	7	3	2400M	1M	2,10,2	7
Bank Mktg. (BM)	45K	6	4	2K	110K	4,5,2,2	11

support multidimensional predicates and range queries via dyadic decomposition. However, the two approaches differ fundamentally in their estimation strategy. OmniSketch represents data implicitly as samples of tuple identifiers and answers queries by computing intersections of these samples at query time. In contrast, PACHA SKETCHES directly maintains aggregate information over multidimensional regions and answers queries via direct sketch aggregation, without requiring explicit set intersections. As a result, OmniSketch incurs additional overhead due to sample maintenance and intersection operations, whereas PACHA SKETCHES avoids these costs and provides more direct control over error propagation.

B. Experimental Setup

Implementation We implemented PACHA SKETCHES in Java (JDK 1.19) and evaluated it on a machine with an Intel Xeon Silver 4214 CPU, 256 GB RAM, and Ubuntu 24.04. As the state-of-the-art baseline, we used the recently proposed Omni Sketch [30] and the authors' provided implementation (also in Java). We only did the following minor modifications to the original code: (i) we isolated the components needed for Omni Sketch using Kmin sampling, as presented in the paper; (ii) we modified the code to support point and range queries at the same time. Their code already provides the implementation for both types of queries, we combined them by mantaining two separate sketch arrays, one containing simple count-min sketches for point queries, and another one containing a set of dyadic count-min sketches for range queries; (iii) we modified the methods to compute dyadic indices and ranges to support negative values, as the original implementation only supported positive values. Although sampling-based methods do not cover all our assumptions and requirements (see Sections I and II), we also implemented Priority Sampling [21] as an alternative baseline for completeness. As the ground-truth baseline, we used Pandas 2.2.2 to answer multidimensional aggregate queries. Our PACHA SKETCH implementation, the Omni Sketch and Priority Sampling implementations used, query workloads, and evaluation scripts are publicly available[2].

Datasets. We evaluate PACHA SKETCHES on four datasets, summarized in Table II. The first is the *Lineitem* table from the TPC-H benchmark [45], with ID and *comment* columns removed. The *Tax* and *Discount* attributes are treated as categorical, as they each take only 10 distinct values. By default, we use a scale factor of 1 (6M rows), and for scalability experiments, we vary the scale to produce datasets with 187K, 750K, 3M, 12M, and 48M rows (scale factor 8). The second dataset is *Online Retail* [46], augmented with *age* and *gender*. The third is the *US Census* dataset [47], filtered to records from California in 2017, with 10 selected demographic columns. The fourth is the *Bank Marketing* dataset [48], comprising customer responses to marketing campaigns. We encoded all date fields as integers, representing the number of days since the earliest date in each dataset.

Query Workload. As discussed in Section IV-C, prior work suggests that real-world analyses often limit queries to four dimensions when reconstructing probability distributions [38], [39]. To better capture realistic workloads, we use LLMs to rank the columns of each dataset by their relevance for analysis. This ranking is then used to guide predicate selection via a weighted distribution. Specifically, given an ordered list of columns, the sampling probability of the i-th column is proportional to 0.5^i, biasing selection toward higher-ranked columns.

Based on this, we generate 200 queries per dataset, each with up to four predicates. For each query, the total number of predicates k is chosen uniformly at random from $\{1, \ldots, 4\}$. Given k, we select the number of numerical predicates k_{num} uniformly at random from $\{0, \ldots, k\}$ and set the number of categorical predicates to $k_{\text{cat}} = k - k_{\text{num}}$. Predicate columns are then sampled without replacement according to the weighted distribution.

For categorical predicates, values are drawn uniformly at random from the domain of the selected column. For numerical predicates, we generate range queries by sampling a range center uniformly from the data domain and constructing a symmetric interval with width approximately 25% of the interquartile range (IQR) of the column. This procedure yields queries with a mix of low- and moderate-selectivity predicates (median selectivity 7%, 25th percentile 3%, and 75th percentile 15%).

For comparison, we also generate a uniform random workload in which predicate columns are selected uniformly at random without weighting.

In addition, we construct four controlled workloads that systematically vary (i) tuple selectivity, (ii) the number of categorical predicates, (iii) the number of numerical predicates, and (iv) the number of mixed predicates.

In contrast to the query workload used in the OmniSketch publication [30], which focuses on uniformly generated point queries, our workload introduces data-dependent range sizes. It explicitly controls predicate composition, yielding a broader, more heterogeneous set of queries.

In total, we generate 19,000 queries across all datasets and workloads.

Error Metrics. We evaluate the estimation accuracy of PACHA SKETCHES using: (i) *normalized error* as $\frac{|\hat{v}-v|}{n}$, where $\hat{v}$ is the estimated result, v the true result, and n is the number tuples; (ii) *relative error* as $\frac{|\hat{v}-v|}{v}$; and (iii) *relative entropy* (Kullback-Leibler (KL) divergence [49]), for measuring the similarity between true and estimated value distributions across queries.

[2]https://github.com/rudip7/PachaSketch

Efficiency Metrics. We measure the efficiency in terms of update *throughput* as the number of tuples updated per second, and median *query latency* to answer a multidimensional aggregate query. We also measure the *memory footprint* of PACHA SKETCHES and Omni Sketches, defined as the total size of all data structures used to answer queries, in MB, using the theoretical formulas from both publications.

Default Parameters. Unless otherwise specified, we use the bases from Table II, $L = 5$, Bloom filters with $p = 0.01$, and Count-Min sketches with $\delta = 0.01$ and $\varepsilon = 0.0001$. We limit $\mathbb{B}$-adic cubes to 1M before coarsening predicates and materialize all single attributes, all pairs, and the full attribute space; for Ltm and BM, we additionally materialize the most queried triplet (see Table II). For Omni Sketches, we use the original parameters ($\delta = 0.1$, $\varepsilon = 0.1$) and adjust B to match PACHA SKETCHES's memory footprint. Higher-accuracy settings increase Omni's conservative-fallback threshold quadratically and consistently yielded worse estimates; thus, we use the original recommended configuration. Its δ and ε are not directly comparable to PACHA SKETCHES's due to the different sketching mechanisms (Table I).

C. Accuracy Different Aggregates

We begin by comparing the accuracy of the aggregate functions supported by PACHA SKETCHES, aiming to show that its estimates are consistent across these functions.

Setup. We used the Lineitem dataset and, similar to TPCH Query 6 [45], used *l_extendedprice * l_quantity* as the aggregate attribute. For each aggregate function, we constructed a PACHA SKETCH instance with the default parameters from Section V-B and executed the random query workload.

Results. Figure 5 shows the accuracy for the different aggregates. PACHA SKETCHES achieves low relative error across all functions, with median values below 10%, even though count-min sketches are not designed to guarantee tight relative error. More importantly, the estimated distributions closely match the ground truth, with relative entropy near zero. Note that min and max are more sensitive to hash collisions, as they are non-linear, unlike sum and count. Given the Zipfian distribution of the aggregated attribute, collisions on high values degrade the accuracy of the max aggregate, while min remains more accurate. Based on these results, the rest of the evaluation focuses on count queries unless stated otherwise, as these are the original use case for count-min sketches.

Summary. PACHA SKETCHES accurately estimates different aggregates.

D. Accuracy vs. SOTA

We compare the accuracy of PACHA SKETCHES (PS) against the state-of-the-art Omni Sketch [30] (OS) across multiple datasets to evaluate (i) the robustness of PACHA SKETCH's accuracy across data distributions and dimensionalities, and (ii) its accuracy under different query workloads.

Setup. We used all datasets in Table II. For each, we constructed a PACHA SKETCH and an Omni Sketch with default parameters from Section V-B, ensuring similar memory footprints, and executed the workload described in Section V-B.

Random Queries. Figure 6 shows the results. Boxplots in Figure 6a display the normalized error per dataset. PACHA SKETCHES consistently outperforms Omni Sketches by one to three orders of magnitude. Omni Sketches shows a smaller improvement for Lineitem due to its many numerical attributes and materialized subspaces, which increase the number of PACHA SKETCH updates and memory usage. With more memory, Omni Sketches improve, but the relative entropy of their estimates (red crosses) still lags behind PACHA SKETCHES, which remains close to zero, indicating near-perfect distribution estimates. Figure 6b details how normalized error grows with the number of query regions, confirming the theoretical analysis in Section IV-C; the slope depends on the dataset and distribution. In practice, query coarsening controls PACHA SKETCHES' memory usage while maintaining low estimation error. For Lineitem, 8% of random queries triggered coarsening at level 3, introducing an average normalized error of only 0.004 and accounting for 98% of the total error. Without coarsening, the Java implementation runs out of memory.

Workload Variations. As noted earlier, the only Omni Sketch configuration that yields reasonable estimates is for the Lineitem dataset, so we focus on this dataset to evaluate the accuracy of both approaches under different query workloads (Figure 7). Across all selectivity levels (Figure 7a), PACHA SKETCHES outperforms Omni Sketches by at least two orders of magnitude. Both remain stable up to 16% selectivity, after which Omni Sketch accuracy drops as cell samples become less representative.

For point queries on categorical attributes (Figure 7b), Omni Sketches meet their guarantees, while PACHA SKETCHES achieves perfect accuracy by updating counts of these regions at the base sketch from the highest $\mathbb{B}$-adic level. At higher levels of the $\mathbb{B}$-adic hierarchy, multiple fine-grained values are aggregated into coarser cells, so the number of distinct cells decreases exponentially. Intuitively, instead of tracking many individual values, the sketch operates over a small number of aggregated groups, thereby reducing collisions and improving accuracy.

For range queries on numerical attributes (Figure 7c), Omni Sketch accuracy improves with lower selectivity due to uniformity assumptions, while PACHA SKETCHES's accuracy depends on the number of query regions; more predicates create more $\mathbb{B}$-adic combinations, and the 1M cube cap limits some queries.

Finally, for mixed queries (Figure 7d), Omni Sketches behave similarly to range queries, whereas PACHA SKETCHES uses categorical predicates to filter out unpopulated regions, improving accuracy and achieving exact results.

Summary. PACHA SKETCHES significantly outperforms the state-of-the-art Omni Sketches across all datasets, workloads, and configurations.

E. Efficiency vs. SOTA

We now evaluate the efficiency of PACHA SKETCHES.

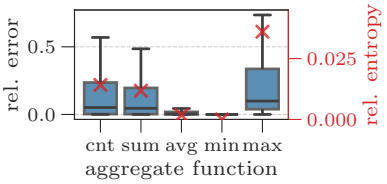
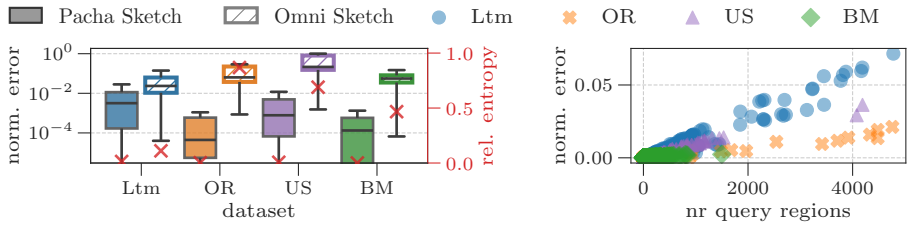


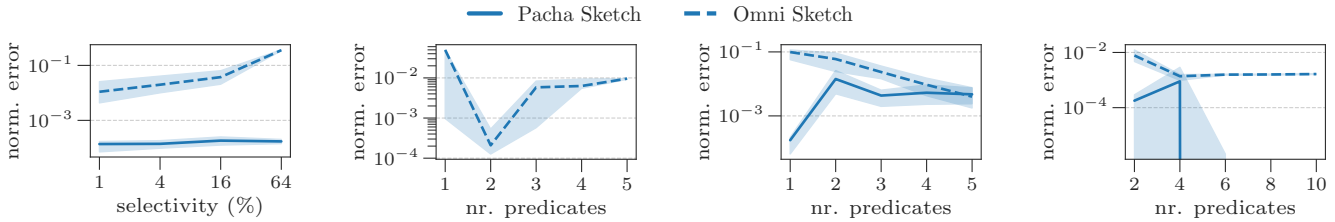
Fig. 5: Aggregate accuracy.



(a) Normalized error

(b) Pacha S. error analysis

Fig. 6: Accuracy across datasets for random queries.



(a) Increasing selectivity

(b) Categorical predicates

(c) Numerical predicates

(d) Mixed predicates

Fig. 7: Accuracy against the state of the art across query workloads.

TABLE III: Ablation study on Lineitem.

	all_idx	no_idx	no_cat_idx	no_num_idx	no_reg_idx
query regions	735.50	21940.49	735.50	735.61	1366.91
norm. error	0.0098	0.3903	0.0098	0.0099	0.0197
runtime	15.56	40.63	24.85	32.96	10.25

Setup. We use the previous setup and, in addition, a parallel version (PSs) of PACHA SKETCHES, in which updates to a single materialized subspace are parallelized per thread. We use the same number of threads as subspaces (Table II).

Update Efficiency. Figure 8a shows average update throughput for both approaches and the additional PACHA SKETCH configuration. Omni Sketches achieve higher throughput than PACHA SKETCHES for all datasets except Lineitem. As discussed in Section V-D, Omni Sketches' smaller memory footprint allows high throughput: once samples in each cell are filled, further sampling stops. For Lineitem, PACHA SKETCHES performs comparably, as multiple subspaces are materialized (Table II). For example, Lineitem has 13 subspaces; each requires $(|t_{cat}|+1)+L+2(|t_{cat}|+1)\cdot L$ internal updates (Section IV-A), totaling 71 updates per subspace and 923 overall. However, since PACHA SKETCHES are mergeable and updates across subspaces are independent, they can be efficiently parallelized [50]. With this configuration, PACHA SKETCHES achieves an 11× higher throughput than Omni Sketches for Lineitem.

Effective pruning depends on the index structures, so we ran an ablation (Table III) on the lineitem dataset to confirm our findings from Fig. 6b and Figure 8c.

Query Efficiency. Figure 8b shows query latency on the random workload. PACHA SKETCHES achieves a median latency below 5 ms across all datasets, whereas Omni Sketches are up to 555× slower (Lineitem) due to complex sample intersections and larger samples. Figure 8c details latency for PACHA SKETCH, showing a strong correlation between internal queries (filters and base sketches) and total latency. Pruning unpopulated regions keeps the number of internal queries manageable, preventing exponential blowup (Section IV-B).

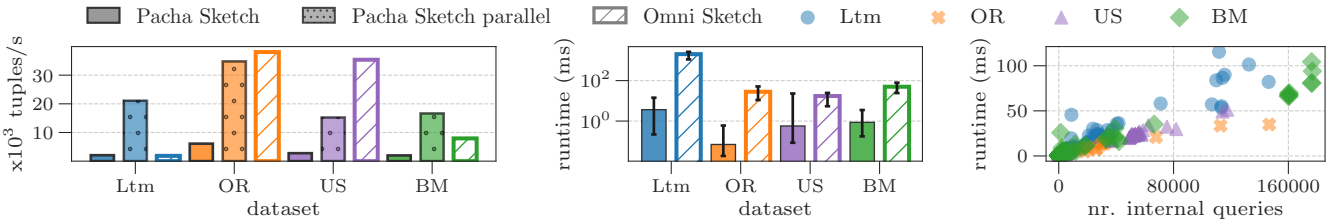
Summary. The parallel design of PACHA SKETCHES enables high update throughput, and its query algorithm answers multidimensional aggregate queries in SOTA.

F. Accuracy vs. Sampling

The data mining literature distinguishes sampling-based techniques from sketch-based summaries, as they target different accuracy, latency, and robustness objectives [51], making direct comparisons challenging. To provide a comprehensive evaluation of PACHA SKETCH, we compare it against Priority Sampling [21], a state-of-the-art sampling-based approach for count and sum queries. Our goal is to evaluate how PACHA SKETCHES compares with sampling-based techniques in terms of accuracy within a given latency budget.

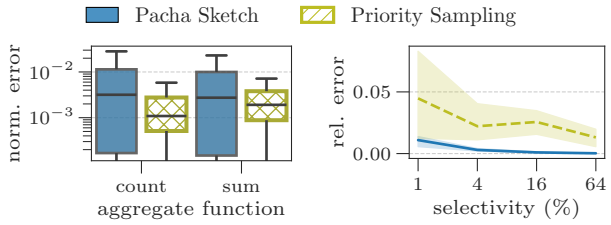
Setup. For a fair comparison, we chose the sample size (15k) for Priority Sampling so that its query latency matches the median latency of PACHA SKETCHES on the random query workload for the Lineitem dataset using default parameters. We then executed the random queries and the increasing-target-selectivity workload, and repeated the experiment for count and sum queries.

Results. Figure 9a shows accuracy for both approaches. PACHA SKETCHES achieves accuracy comparable to Priority Sampling for both aggregates, but exhibits higher variance under the random query workload. This is because PACHA SKETCHES's accuracy depends on the number of internal queries performed (Figure 6b), which varies across queries. In contrast, the queries in the increasing selectivity workload



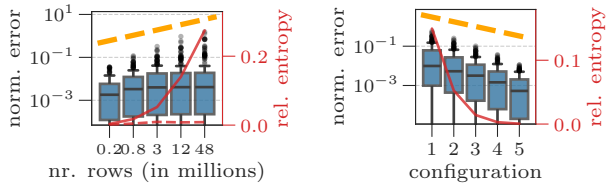
(a) Update efficiency (b) Query efficiency (c) Efficiency analysis

Fig. 8: Update and query efficiency across different datasets.



(a) Normalized error (b) Increasing selectivity

Fig. 9: Accuracy of PACHA SKETCHES (PS) compared to Priority Sampling (S) for different aggr. and selectivities.



(a) Increasing number of rows. (b) Accuracy guarantees.

Fig. 10: Accuracy increasing the number of rows and accuracy guarantees (see Table IV).

TABLE IV: Configurations used for the experiments on increasing accuracy guarantees.

Config	p	ε	δ	Size MB
1	0.04	0.0004	0.01	231.9
2	0.02	0.0002	0.01	314.9
3	0.01	0.0001	0.01	440.0
4	0.005	0.00005	0.01	649.4
5	0.001	0.00001	0.01	1027.3

(Fig. 9b) align more with the $\mathbb{B}$-adic space, resulting in stable accuracy for PACHA SKETCHES (median relative error 1%), outperforming Priority Sampling (median relative error 4.4%). At higher selectivities, Priority Sampling variance increases, whereas PACHA SKETCHES remains low, as filtering prunes unpopulated regions and maintains estimate quality independent of selectivity.

Summary. PACHA SKETCHES competes with the state-of-the-art in sampling methods for aggregate estimation, while providing lower variance for high-selectivity queries.

G. Increasing Accuracy Guarantees

We now investigate the scalability of PACHA SKETCH's accuracy guarantees. Our goal is to show that PACHA SKETCHES maintains high accuracy and scales gracefully for higher accuracy requirements.

Setup. We used the Lineitem SF 0.1 dataset with the random query workload and constructed PACHA SKETCH instances with increasing accuracy guarantees (Table IV). The memory footprint of each instance was measured from actual instantiated structures. The formula from Sec. IV-C is only a simplified version, which does not account for the number of materialized numerical subspaces and the repeated insertions into the base sketches.

Results. Figure 10b shows the accuracy for each configuration. Tighter accuracy guarantees significantly improve PACHA SKETCHES's estimates, reaching near-perfect accuracy for configuration 5. Increasing guarantees increases the size of internal Bloom filters and Count-Min sketches, reducing hash collisions and improving estimates. This comes at the cost of higher memory, roughly 1GB for configuration 5, which is still manageable on modern servers. The orange dashed line shows an approximate theoretical accuracy bound, calculated by multiplying the 75th percentile of internal queries by ε (Section IV-C). The normalized error remains below this bound, confirming that the theoretical analysis holds in practice. Even configuration 1 achieves remarkably low relative entropy.

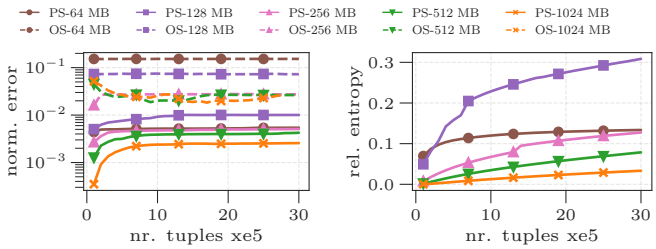
Summary. PACHA SKETCHES gracefully scales its accuracy guarantees, achieving near-perfect accuracy.

H. Increasing Dataset Size

We study the scalability of PACHA SKETCHES as data volumes increase, evaluating whether it maintains its accuracy guarantees for millions of tuples.

Setup. We used the Lineitem dataset and varied the scale factor to generate datasets with 187K, 750K, 3M, 12M, and 48M rows. For each dataset, we constructed a PACHA SKETCH instance with default parameters and used the random query workload.

Results. Figure 10a shows accuracy across dataset sizes. The normalized error remains below the approximated accuracy guarantee bound (orange dashed line), confirming that PACHA SKETCHES maintains its guarantees even at large scale. We also report two measures of relative entropy. The solid red line shows that relative entropy grows linearly with dataset size, as larger datasets introduce more rare attribute combinations and hash collisions. This mainly affects queries with very low selectivity. After removing outliers using IQR filtering, relative



(a) Pacha vs Omni Sketch (b) Rel. entropy Pacha Sketch

Fig. 11: Streaming accuracy with memory constraints.

entropy remains nearly constant (red dashed line). In practice, such outliers are easy to detect, particularly when visualizing estimated distributions.

Summary. PACHA SKETCHES scale to datasets with millions of tuples while maintaining its accuracy guarantees.

I. Data Streaming with Memory Constraints

In this experiment, we evaluate whether PACHA SKETCHES can provide accurate estimates under constrained memory and unbounded input size.

Setup. We initialized PACHA SKETCHES and Omni Sketches with memory budgets of 64 MB, 128 MB, 256 MB, 512 MB, and 1 GB. For each budget, we tuned PACHA SKETCHES to achieve the best possible accuracy for an unbounded dataset. For Omni Sketches, we kept the default parameters and increased the maximum sample size to utilize available memory fully. To simulate a data stream, we used the Lineitem dataset with a scale factor of 8 (48M rows) and streamed the data in chunks of 100K rows. After each chunk, we executed the random query workload and reported the median accuracy.

Results. Figure 11 shows accuracy across memory budgets; due to convergence, we report results only up to 3M tuples. Figure 11a compares PACHA SKETCHES and Omni Sketches. Even with the smallest budget (64MB), PACHA SKETCHES provides more accurate estimates than Omni Sketches with the largest budget (1GB). This is because PACHA SKETCHES limits the maximum number of $\mathbb{B}$-adic cubes based on the memory budget (e.g., 1K for 64MB and 25K for 128MB), forcing more frequent range predicate adjustments. This reduces collisions at higher levels and improves accuracy, at the cost of increased uniformity in the estimated distribution, which slightly degrades relative entropy (Figure 11b).

Summary. PACHA SKETCHES delivers accurate estimates under tight memory constraints, significantly outperforming the SOTA.

J. Impact of PACHA SKETCH *Heuristics*

In Section IV-C, we discussed the heuristics PACHA SKETCHES uses to avoid exponential blowup and improve accuracy and efficiency. In these experiments, we evaluate the impact of these heuristics to assess their contribution and provide guidance on configuring them. The heuristics include: (i) the attribute order in the AD-tree for the categorical mapping; (ii) the maximum number of allowed $\mathbb{B}$-adic cubes to force range predicate adjustments; and (iii) the selection of the projected numerical subspaces to maintain.

1) Impact of the AD-Tree Attribute Order

Setup. We evaluate the impact of AD-tree ordering using two query workloads: predicates selected via a relevance-weighted distribution (Section V-B), and predicates selected uniformly at random. All queries include at least one categorical predicate, making the AD-tree order relevant for accuracy. We construct two PACHA SKETCH instances: a custom AD-tree ordered by attribute relevance, and a default AD-tree ordered by increasing attribute cardinality. Experiments are conducted on TPC-H Lineitem ($\sim$4K categorical combinations) and US Census ($\sim$2400M combinations).

Results. Figure 12 summarizes the results. On Lineitem, AD-tree order has negligible impact (Fig. 12a). Both configurations perform similarly under the weighted workload. Under the uniform workload, the default order maintains accuracy, whereas the custom order yields $\sim 5\times$ higher error and relative entropy, although the median normalized error remains below 10^{-2}. Runtime trends (Fig. 12b) reinforce that fewer queried regions improve both efficiency and accuracy.

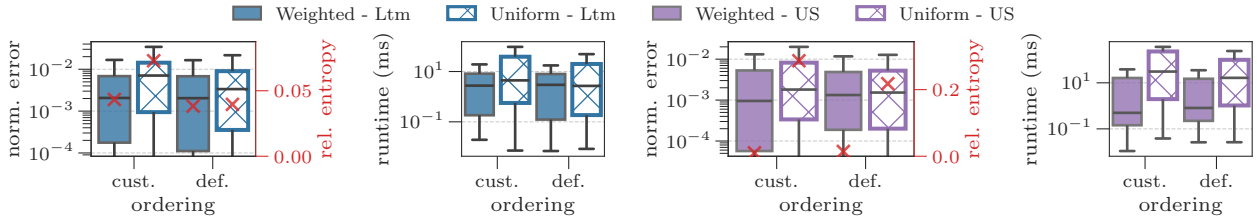
On the US Census, where sparsity is higher, pruning becomes more critical. The custom order achieves better accuracy under the weighted workload (Fig. 12c), but the default order performs better under the uniform workload due to lower branching. As shown in Fig. 12d, uniform workloads map to substantially more regions. While categorical index pruning mitigates this, increased false positives lead to higher relative entropy.

Summary. When the workload is known, configuring the AD-tree based on attribute relevance can improve accuracy. Otherwise, cardinality-based ordering yields more robust performance.

2) Impact of Limiting the Number of $\mathbb{B}$-adic Cubes

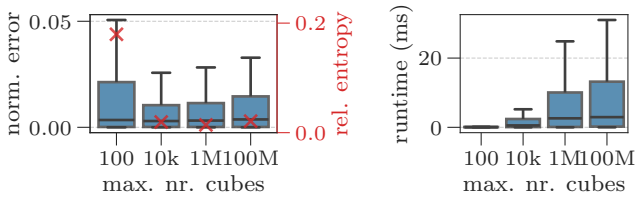
Setup. We evaluate the impact of query coarsening induced by limiting the number of $\mathbb{B}$-adic cubes in the spatial cover, as described in Section IV-C. We use the random query workload on the Lineitem dataset and construct multiple PACHA SKETCH instances with varying maximum cube limits, ranging from 100 to 100M.

Results. Figure 13 shows the results. With a low cube limit (100), many queries are coerced to higher levels in the $\mathbb{B}$-adic hierarchy, resulting in more uniform estimates and higher relative entropy (Fig. 13a). Despite this, the normalized error remains low due to the sparsity of the high-dimensional data. As the cube limit increases, fewer queries are coerced, improving estimate resolution. However, as discussed in Section V-D (Fig. 6b), accuracy correlates with the number of queried regions. Beyond a certain threshold (10M), many small regions are queried using base sketches from the lowest levels, which handle a larger number of distinct values, thereby increasing hash collisions and degrading accuracy. Finally, runtime increases with the cube limit as more regions are queried, but stabilizes beyond 1M cubes due to early pruning of unpopulated regions (Fig. 13b).



(a) Accuracy Ltm. dataset (b) Runtime Ltm. dataset (c) Accuracy US. dataset (d) Runtime US. dataset

Fig. 12: Effect of attribute ordering in the AD-tree on the accuracy and runtime of PACHA SKETCHES.



(a) Normalized error (b) Efficiency

Fig. 13: Effect of the maximum number of allowed $\mathbb{B}$-adic cubes on the accuracy and runtime of PACHA SKETCHES.

Summary. The cube limit is a key parameter for balancing accuracy and efficiency. We recommend setting it based on data sparsity and the expected query workload. In practice, values between 100K and 1M provide a good trade-off in many scenarios.

3) Impact of Subspace Selection

Setup. We evaluate the impact of subspace selection using both weighted and uniform random query workloads on the Lineitem dataset. We construct multiple PACHA SKETCH instances with different sets of materialized subspaces. All configurations include the base subspace over all numerical attributes. We then vary additional subspaces, including single attributes, all pairs, and all combinations of the five numerical attributes. Our custom configuration selects subspaces based on attribute relevance, resulting in 13 subspaces: all single attributes, six pairs, one triplet, and the full subspace.

Results. Figure 14 shows the results. The worst performance is observed when no additional subspaces are materialized (Fig. 14a), as all queries are answered using the $\mathbb{B}$-adic cover over all numerical attributes. Since the workload contains queries with up to four predicates—often not all numerical—the $\mathbb{B}$-adic cover frequently reaches worst-case behavior, leading to an exponential blowup and forcing coarsening to higher levels due to the cube-limit heuristic.

Adding single-attribute subspaces significantly improves accuracy, as many queries can be answered more efficiently. However, queries involving multiple numerical attributes still require falling back to the base subspace, resulting in high variance and relatively poor relative entropy despite improved average accuracy.

Beyond this point, all configurations achieve similar accuracy, as most queries involve at most two numerical attributes. However, the custom configuration exhibits worse relative entropy under the uniform workload. This is because it does not include all attribute pairs, and thus some unforeseen predicate combinations require querying more general subspaces with lower accuracy.

Finally, runtime remains relatively stable across configurations (Fig. 14b), as the cube-limit heuristic prevents an exponential blowup and keeps the number of queried regions manageable.

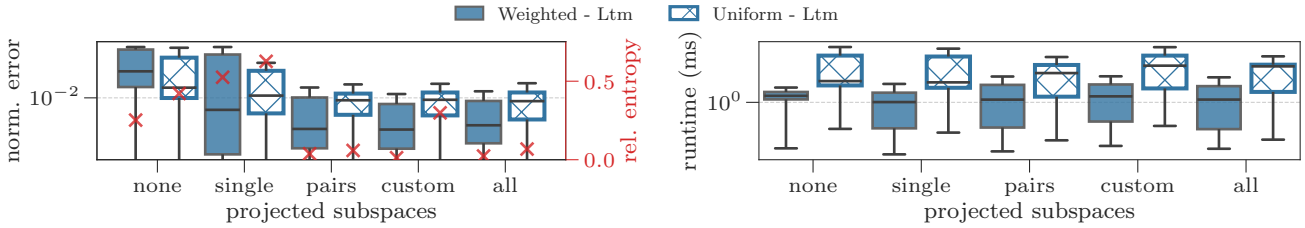
Summary. Selecting subspaces based on the expected workload can significantly improve accuracy. In practice, materializing all single attributes and pairs provides a strong baseline for most realistic workloads [38], [39].

K. *Impact of* PACHA SKETCH *Parameters*

Finally, we investigate the impact of different PACHA SKETCH's parameters on accuracy, aiming to guide the optimal configuration of PACHA SKETCHES for different scenarios.

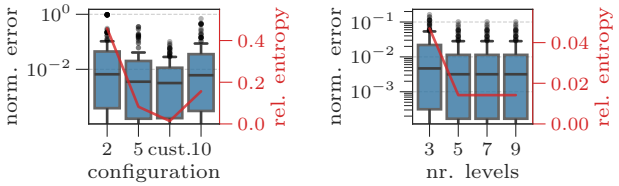
Setup. We used the Lineitem dataset and the random query workload. We first fixed the standard parameters and varied the bases used by the PACHA SKETCH to define the $\mathbb{B}$-adic space of numerical attributes. The numerical columns in Lineitem are *shipdate*, *commitdate*, *receiptdate*, *extendedprice*, and *quantity*. The customized configuration uses bases 5, 5, 5, 10, and 2, respectively. We also evaluated three uniform-base configurations, with all bases set to 2, 5, and 10. Finally, keeping the customized bases fixed, we varied the number of levels L from 3 to 9.

$\mathbb{B}$-adic Bases. Figure 15a shows the accuracy results for the different base configurations. Using only small bases (2) yields worse accuracy because the b-adic cover of each numerical attribute becomes larger, resulting in more query regions (Section III-C). Conversely, using only large bases (10) also degrades accuracy, as mismatches between base levels across different b-adic ranges lead to more downgraded combinations (Section III-C), again increasing the number of query regions. Setting all bases to 5 improves accuracy, but the best results are achieved with a customized configuration informed by attribute domain analysis. For example, *extendedprice* ranges roughly from 900 to 100K, whereas *quantity* ranges from 1 to 50. Using a base of 10 for *extendedprice* enables coarser partitioning, while a base of 2 for *quantity* allows finer-grained partitions.



(a) Normalized error (b) Efficiency

Fig. 14: Effect of the selection of projected numerical subspaces on the accuracy and runtime of PACHA SKETCHES.



(a) Varying $\mathbb{B}$-adic cover bases (b) Varying number of levels

Fig. 15: Accuracy with different parameters.

Number of Levels. Figure 15b shows the accuracy for different values of levels L. Increasing the number of levels improves accuracy by enabling larger $\mathbb{B}$-adic cubes to cover the query space. However, beyond $L = 5$, the gains diminish in our setup, as the workload does not include queries that benefit from additional levels.

Summary. These results highlight the importance of workload characteristics and attribute domain analysis when configuring PACHA SKETCHES. With appropriate parameter choices, PACHA SKETCHES achieves high accuracy while maintaining a small memory footprint.

VI. RELATED WORK

Summarizing joint distributions over many attributes has been studied extensively, but classical approaches suffer from scalability issues. Multidimensional histograms [25], [26] and transform-based summaries, like wavelets [27], capture joint distributions, but their size grows rapidly with dimension. For example, Matias et al. can improve the maintenance costs of simple bucket histograms, but regardless of the sampling optimization, these approaches become prohibitively expensive to maintain for multiple dimensions [27]. Similarly, caching structures such as AD-trees [31] aim to maintain multiple statistics, but their memory consumption grows with increasing dimensionality. In general, traditional solutions either grow in size or lose accuracy dramatically as dimensionality increases.

Sketch-based summaries, like Count-min sketches [11], aim to bound the memory footprint at the price of accuracy. However, we want to retain the high-dimensional data frequencies. In that case, the accuracy drops significantly due to the curse of dimensionality, which makes the data distribution increasingly sparse and uniform with each additional dimension. Additionally, sketch-based summaries would require iterating over all points to compute frequencies, which is infeasible in high-dimensional settings. More modern approaches, such as Hydra [29] and Omni Sketch [30], have recently sought to address this issue. However, Hydra's approach is not scalable [30], as its update time and memory footprint grow with the number of dimension combinations (2^d). On the other hand, Omni Sketches combines sketching and sampling techniques to improve the accuracy of multidimensional queries.

A recent line of work has proposed using learned models to create approximate sketches [52] with flexible support for predicates and relations. These approaches train models using dyadic ranges as features to capture distributional properties for range queries better. We believe that our multidimensional data organization can complement these learned sketching techniques by providing a structured way to decompose interactions across categorical and numerical dimensions, improving the model's ability to represent complex predicate interactions.

Finally, sampling and approximate query processing (AQP) approaches [20]–[24] have been proposed to cap memory usage while preserving the expressiveness of raw data. However, these approaches can heavily bias results, especially for multidimensional predicates and selective queries, as outliers in the sample can skew the results.

VII. CONCLUSION

In this paper, we introduced a hierarchical organization of mixed-type multidimensional spaces that distinguishes between categorical and numerical dimensions. By decomposing the domain into an AD-tree over categorical attributes and a $\mathbb{B}$-adic hierarchical range cover over numerical attributes, this organization provides a reusable abstraction for scalable computation over high-dimensional data, enabling efficient predicate composition and compact representation of joint distributions.

As an instantiation of this organization, we presented PACHA SKETCHES, a modular sketching framework for multidimensional aggregate queries. Leveraging the proposed decomposition, PACHA SKETCHES supports equality predicates over categorical attributes and range predicates over numerical attributes, while providing provable guarantees on accuracy and space. PACHA SKETCHES is also naturally parallelizable and mergeable, making it well-suited for distributed, streaming, and federated environments. Our evaluation showed that

PACHA SKETCHES scales to high-dimensional datasets under tight memory budgets and consistently outperforms state-of-the-art methods by up to three orders of magnitude in estimation accuracy and two orders of magnitude in query performance.

Beyond approximate aggregation, the proposed hierarchical organization is broadly applicable. By exposing a structured decomposition of mixed-type multidimensional spaces, it can support multiresolution feature construction in machine learning, principled discretization for representation learning, and new classes of multidimensional indexing techniques that exploit categorical-numerical asymmetries [52]–[56].

ACKNOWLEDGMENT

We gratefully acknowledge funding from the German Federal Ministry of Education and Research under the grant BIFOLD26B. The work of Kaustubh Beedkar is supported by Anusandhan National Research Foundation project ANRF/ECRG/2024/005781/ENS.

AI-GENERATED CONTENT ACKNOWLEDGMENT

The authors used OpenAI's ChatGPT to assist with the preparation of this manuscript and the supporting experimental artifacts. In particular, ChatGPT was used for language editing, grammar improvement, and phrasing suggestions, as well as for assistance with developing code for visualizing experimental results and with implementing additional coding infrastructure to facilitate the execution and management of experiments. The authors reviewed, validated, and revised all AI-assisted outputs and remain fully responsible for the accuracy, originality, and final content of the manuscript, code, and experimental results.

REFERENCES

[1] L. X. Clegg, B. F. Hankey, R. Tiwari, E. J. Feuer, and B. K. Edwards, "Estimating average annual per cent change in trend analysis," *Statistics in medicine*, vol. 28, no. 29, pp. 3670–3682, 2009.

[2] A. Hess, H. Iyer, and W. Malm, "Linear trend analysis: a comparison of methods," *Atmospheric environment*, vol. 35, no. 30, pp. 5211–5222, 2001.

[3] B. Krishnamurthy, S. Sen, Y. Zhang, and Y. Chen, "Sketch-based change detection: Methods, evaluation, and applications," in *Proceedings of the 3rd ACM SIGCOMM conference on Internet measurement*, 2003, pp. 234–247.

[4] A. Soule, K. Salamatian, and N. Taft, "Combining filtering and statistical methods for anomaly detection," in *Proceedings of the 5th ACM SIGCOMM conference on Internet Measurement*, 2005, pp. 31–31.

[5] D. Cai, M. Mitzenmacher, and R. P. Adams, "A bayesian nonparametric view on count-min sketch," *Advances in neural information processing systems*, vol. 31, 2018.

[6] S. Kashyap, S. Gombar, S. Yadlowsky, A. Callahan, J. Fries, B. A. Pinsky, and N. H. Shah, "Measure what matters: counts of hospitalized patients are a better metric for health system capacity planning for a reopening," *Journal of the American Medical Informatics Association*, vol. 27, no. 7, pp. 1026–1131, 2020.

[7] Z. Lin, E. Barrena, and R. S. Kwan, "Train unit scheduling guided by historic capacity provisions and passenger count surveys," *Public Transport*, vol. 9, no. 1, pp. 137–154, 2017.

[8] S. R. Chalamala, N. K. Kummari, A. K. Singh, A. Saibewar, and K. M. Chalavadi, "Federated learning to comply with data protection regulations," *CSI Transactions on ICT*, vol. 10, no. 1, pp. 47–60, 2022.

[9] K. Beedkar, J.-A. Quiané-Ruiz, and V. Markl, "Navigating compliance with data transfers in federated data processing." *IEEE Data Eng. Bull.*, vol. 45, no. 1, pp. 50–61, 2022.

[10] ——, "Compliant geo-distributed query processing," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 181–193.

[11] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[12] S. Heule, M. Nunkesser, and A. Hall, "Hyperloglog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm," in *Proceedings of the 16th International Conference on Extending Database Technology*, 2013, pp. 683–692.

[13] H. Samet, "The quadtree and related hierarchical data structures," *ACM Computing Surveys (CSUR)*, vol. 16, no. 2, pp. 187–260, 1984.

[14] D. Meagher, "Geometric modeling using octree encoding," *Computer graphics and image processing*, vol. 19, no. 2, pp. 129–147, 1982.

[15] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow, and H. Pirahesh, "Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals," *Data mining and knowledge discovery*, vol. 1, no. 1, pp. 29–53, 1997.

[16] G. Cormode, M. Garofalakis, P. J. Haas, C. Jermaine *et al.*, "Synopses for massive data: Samples, histograms, wavelets, sketches," *Foundations and Trends® in Databases*, vol. 4, no. 1–3, pp. 1–294, 2011.

[17] V. Poosala, *Histogram-based estimation techniques in database systems*. The University of Wisconsin-Madison, 1997.

[18] F. Rusu and A. Dobra, "Fast range-summable random variables for efficient aggregate estimation," in *Proceedings of the 2006 ACM SIGMOD international conference on Management of data*, 2006, pp. 193–204.

[19] ——, "Pseudo-random number generation for sketch-based estimations," *ACM Transactions on Database Systems (TODS)*, vol. 32, no. 2, pp. 11–es, 2007.

[20] S. Agarwal, B. Mozafari, A. Panda, H. Milner, S. Madden, and I. Stoica, "Blinkdb: queries with bounded errors and bounded response times on very large data," in *Proceedings of the 8th ACM European conference on computer systems*, 2013, pp. 29–42.

[21] N. Duffield, C. Lund, and M. Thorup, "Priority sampling for estimation of arbitrary subset sums," *Journal of the ACM (JACM)*, vol. 54, no. 6, pp. 32–es, 2007.

[22] D. Ting, "Data sketches for disaggregated subset sum and frequent item estimation," in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 1129–1140.

[23] A. Metwally, D. Agrawal, and A. El Abbadi, "Efficient computation of frequent and top-k elements in data streams," in *International conference on database theory*. Springer, 2005, pp. 398–412.

[24] E. Cohen, N. Duffield, H. Kaplan, C. Lund, and M. Thorup, "Sketching unaggregated data streams for subpopulation-size queries," in *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, 2007, pp. 253–262.

[25] N. Thaper, S. Guha, P. Indyk, and N. Koudas, "Dynamic multidimensional histograms," in *Proceedings of the 2002 ACM SIGMOD international conference on Management of data*, 2002, pp. 428–439.

[26] M. Muralikrishna and D. J. DeWitt, "Equi-depth multidimensional histograms," in *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*, 1988, pp. 28–36.

[27] Y. Matias, J. S. Vitter, and M. Wang, "Dynamic maintenance of wavelet-based histograms," in *VLDB*, vol. 2, no. 2.1, 2000, pp. 2–4.

[28] S. Suri, C. D. Tóth, and Y. Zhou, "Range counting over multidimensional data streams," in *Proceedings of the twentieth annual symposium on Computational geometry*, 2004, pp. 160–169.

[29] A. Manousis, Z. Cheng, R. B. Basat, Z. Liu, and V. Sekar, "Enabling efficient and general subpopulation analytics in multidimensional data streams," *arXiv preprint arXiv:2208.04927*, 2022.

[30] W. R. Punter, O. Papapetrou, and M. Garofalakis, "Omnisketch: efficient multi-dimensional high-velocity stream analytics with arbitrary predicates," *arXiv preprint arXiv:2309.06051*, 2023.

[31] A. Moore and M. S. Lee, "Cached sufficient statistics for efficient machine learning with large datasets," *Journal of artificial intelligence research*, vol. 8, pp. 67–91, 1998.

[32] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[33] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of massive data sets*. Cambridge university press, 2020.

[34] D. Vengerov, A. C. Menck, M. Zait, and S. P. Chakkappen, "Join size estimation subject to filter conditions," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1530–1541, 2015.

[35] N. Alon, Y. Matias, and M. Szegedy, "The space complexity of approximating the frequency moments," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, 1996, pp. 20–29.

[36] M. Hay, V. Rastogi, G. Miklau, and D. Suciu, "Boosting the accuracy of differentially-private histograms through consistency," *Proceedings of the VLDB Endowment*, vol. 3, no. 1, pp. 1021–1032, 2010.

[37] W. Qardaji, W. Yang, and N. Li, "Understanding hierarchical methods for differentially private histograms," *Proceedings of the VLDB Endowment*, vol. 6, no. 14, pp. 1954–1965, 2013.

[38] T. Itoh, A. Kumar, K. Klein, and J. Kim, "High-dimensional data visualization by interactive construction of low-dimensional parallel coordinate plots," *Journal of Visual Languages & Computing*, vol. 43, pp. 1–13, 2017.

[39] D. C. Hoaglin, F. Mosteller, and J. W. Tukey, *Understanding robust and exploratory data analysis*. John Wiley & Sons, 2000.

[40] G. Cormode, T. Kulkarni, and D. Srivastava, "Answering range queries under local differential privacy," *Proceedings of the VLDB Endowment*, vol. 12, no. 10.

[41] G. Cormode, F. Korn, S. Muthukrishnan, and D. Srivastava, "Diamond in the rough: Finding hierarchical heavy hitters in multi-dimensional data," in *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*, 2004, pp. 155–166.

[42] M. Charikar, K. Chen, and M. Farach-Colton, "Finding frequent items in data streams," in *International Colloquium on Automata, Languages, and Programming*. Springer, 2002, pp. 693–703.

[43] F. Zhao, D. Qiao, R. Redberg, D. Agrawal, A. El Abbadi, and Y.-X. Wang, "Differentially private linear sketches: Efficient implementations and applications," *Advances in Neural Information Processing Systems*, vol. 35, pp. 12 691–12 704, 2022.

[44] R. Pagh, M. Stöckel, and D. P. Woodruff, "Is min-wise hashing optimal for summarizing set intersection?" in *Proceedings of the 33rd ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, 2014, pp. 109–120.

[45] Transaction Processing Performance Council, "Tpc benchmark h (decision support) standard specification," Transaction Processing Performance Council, Tech. Rep., 2017, accessed: 2024-04-16. [Online]. Available: http://www.tpc.org/tpch/

[46] D. Chen, "Online Retail [dataset]," UCI Machine Learning Repository, 2015, donated on November 5, 2015. Transactional data from 2010-2011 for a UK-based online retail store. [Online]. Available: https://archive.ics.uci.edu/dataset/352/online+retail

[47] F. Ding, M. Hardt, J. Miller, and L. Schmidt, "Retiring adult: New datasets for fair machine learning," pp. 6478–6490, 2021. [Online]. Available: https://proceedings.neurips.cc/paper/2021/hash/32e54441e6382a7fbacbbbaf3c450059-Abstract.html

[48] S. Moro, P. Cortez, and P. Rita, "Bank Marketing [dataset]," UCI Machine Learning Repository, 2012, donated on February 13, 2012. Data from Portuguese bank marketing campaigns (May 2008-November 2010), 45 211 instances, 16-20 features. [Online]. Available: https://archive.ics.uci.edu/dataset/222/bank+marketing

[49] S. Kullback and R. A. Leibler, "On information and sufficiency," *The annals of mathematical statistics*, vol. 22, no. 1, pp. 79–86, 1951.

[50] R. Poepsel-Lemaitre, M. Kiefer, J. Von Hein, J.-A. Quiané-Ruiz, and V. Markl, "In the land of data streams where synopses are missing, one framework to bring them all," *Proceedings of the VLDB Endowment*, vol. 14, no. 10, pp. 1818–1831, 2021.

[51] M. Garofalakis, J. Gehrke, and R. Rastogi, "Querying and mining data streams: you only get one look a tutorial," in *Proceedings of the 2002 ACM SIGMOD international conference on Management of data*, 2002, pp. 635–635.

[52] B. Tsan, A. Datta, Y. Izenov, and F. Rusu, "Approximate sketches," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–24, 2024.

[53] G. Blanchard, C. Schäfer, Y. Rozenholc, and K.-R. Müller, "Optimal dyadic decision trees," *Machine Learning*, vol. 66, no. 2, pp. 209–241, 2007.

[54] S. Van Der Pas and V. Ročková, "Bayesian dyadic trees and histograms for regression," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[55] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013.

[56] H. Samet, *Foundations of multidimensional and metric data structures*. Morgan Kaufmann, 2006.